

Part 1: Introduction to QGIS and Python

1. What is QGIS and its use in geospatial analysis?

What is QGIS?

QGIS, which stands for Quantum GIS, is a free and open-source Geographic Information System software. In simple terms, it's a computer program that lets you view, edit, analyze, and visualize spatial data that is, data that is linked to locations on Earth.

What QGIS offers?

- Create digital maps with different layers,
- Perform spatial analysis such as buffering and overlay,
- Work with satellite images, shape files, and databases,
- And even connect it with Python for automation and advanced analysis.

Usage

- QGIS is widely used in many fields. Let's go through some key applications:
- Urban Planning Planners use QGIS to design city layouts, zoning, and infrastructure.
- Environmental Monitoring Scientists track land use, deforestation, and pollution with QGIS.
- Agriculture Farmers use QGIS to manage land, crops, and soil data.
- Disaster Management Emergency teams map flood zones and vulnerable areas using QGIS.
- Transportation Engineers analyze road networks and traffic flows through QGIS.

Availability

Unlike many other GIS tools, QGIS is completely free.

That means students, researchers, and professionals can use it without paying expensive licenses.

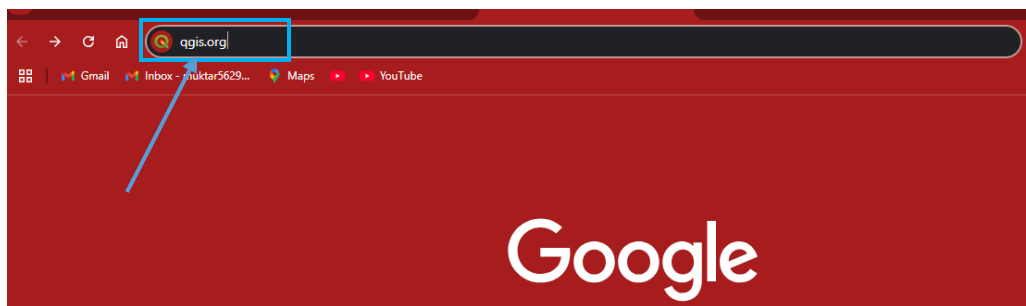
Plus, it has a strong community that constantly improves the software and shares plugins.

2. Installing QGIS and setting up the Python environment (PyQGIS, plugins, etc.)

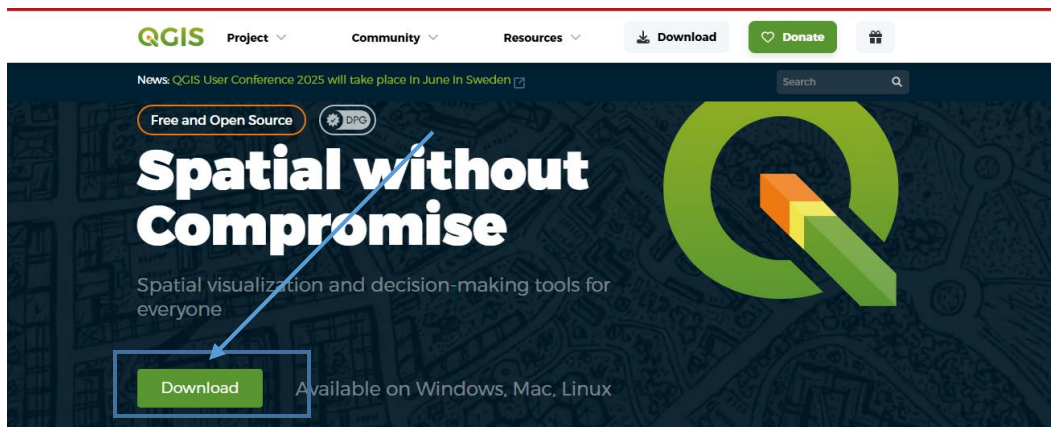
Installing QGIS

Step 1: Go to the QGIS Website

The first thing we need to do is open your web browser and visit the official QGIS website: qgis.org.

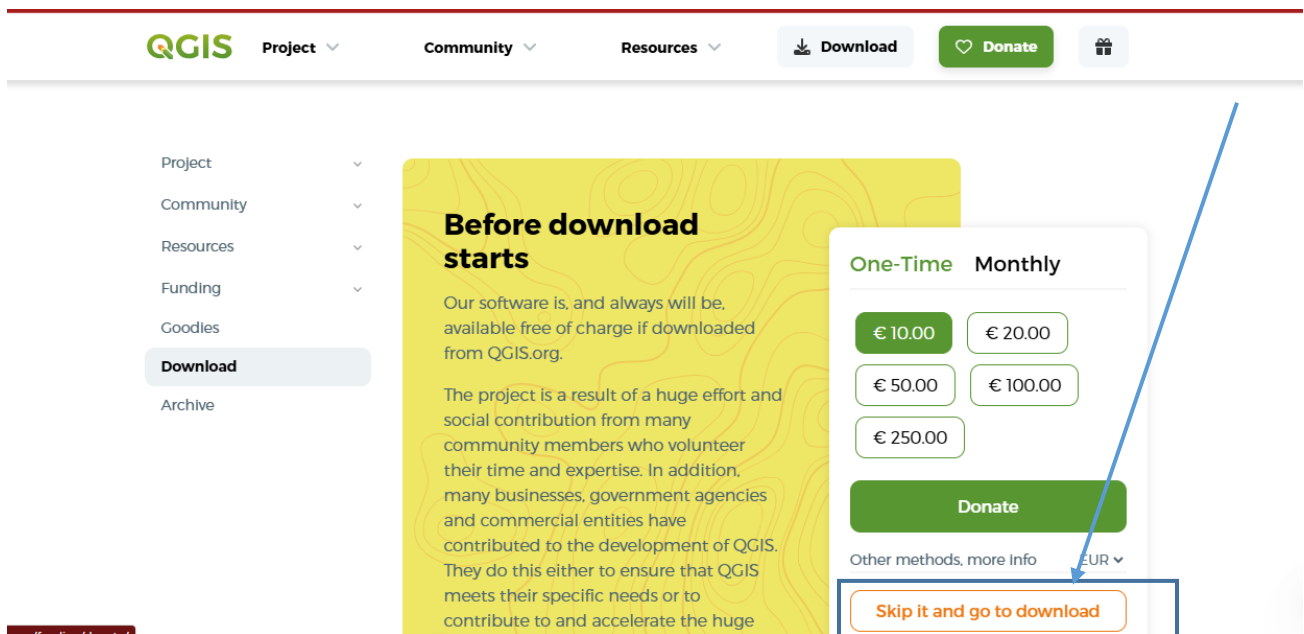


Once you're on the homepage, you should see a Download Now button. Go ahead and click on that button to begin the download process.



Optional Donation

After clicking Download Now, you'll be directed to a donation page. Here, you will be asked if you'd like to support the development of QGIS with a donation. But don't worry! You are not required to donate in order to download QGIS. If you don't want to donate, simply look for the "Skip it and go to download" link located below the donation prompt and click it.



This will take you straight to the download page without any interruptions.



📄 Step 2: Selecting the Right Version for Windows

Now that you've skipped the donation, you will be redirected to the Download QGIS for your platform page.

We'll see that the current version of QGIS is 3.42.2 'Münster', which was released on April 18, 2025. This is the latest version of QGIS, and it's fully equipped with all the newest features.

QGIS

Project

Community

Resources

Download

Donate

	QGISQT6-OSGeo4W-3.40.6-1.sha256sum	2025-04-20 11:16	95
	QGISQT6-OSGeo4W-3.42.0-1.msi	2025-02-23 03:16	1.2G
	QGISQT6-OSGeo4W-3.42.0-1.sha256sum	2025-02-23 03:16	95
	QGISQT6-OSGeo4W-3.42.1-1.msi	2025-03-22 22:30	1.2G
	QGISQT6-OSGeo4W-3.42.1-1.sha256sum	2025-03-22 22:30	95
	QGISQT6-OSGeo4W-3.42.2-1.msi	2025-04-20 10:21	1.2G
	QGISQT6-OSGeo4W-3.42.2-1.sha256sum	2025-04-20 10:21	95
	QGISQT6-OSGeo4W-3.42.2-2.msi	2025-04-21 12:26	1.2G
	QGISQT6-OSGeo4W-3.42.2-2.sha256sum	2025-04-21 12:26	95

Long-Term Release (LTR): If you prefer a more stable version of QGIS that doesn't change as often, you can choose the Long-Term Release (LTR) version, which is 3.40.6 'Bratislava'. This version is ideal if you want stability over having the latest features. If you're unsure which one to choose, I recommend the LTR version for beginners.

QGIS

Project

Community

Resources

Download

Donate

	QGISQT6-OSGeo4W-3.40.4-2.sha256sum	2025-03-22 23:23	95
	QGISQT6-OSGeo4W-3.40.5-1.msi	2025-03-28 12:35	1.2G
	QGISQT6-OSGeo4W-3.40.5-1.sha256sum	2025-03-28 12:35	95
	QGISQT6-OSGeo4W-3.40.6-1.msi	2025-04-20 11:15	1.2G
	QGISQT6-OSGeo4W-3.40.6-1.sha256sum	2025-04-20 11:16	95
	QGISQT6-OSGeo4W-3.42.0-1.msi	2025-02-23 03:16	1.2G
	QGISQT6-OSGeo4W-3.42.0-1.sha256sum	2025-02-23 03:16	95
	QGISQT6-OSGeo4W-3.42.1-1.msi	2025-03-22 22:30	1.2G
	QGISQT6-OSGeo4W-3.42.1-1.sha256sum	2025-03-22 22:30	95
	QGISQT6-OSGeo4W-3.42.2-1.msi	2025-04-20 10:21	1.2G
	QGISQT6-OSGeo4W-3.42.2-1.sha256sum	2025-04-20 10:21	95
	QGISQT6-OSGeo4W-3.42.2-2.msi	2025-04-21 12:26	1.2G

Step 3: Downloading QGIS for Windows

Let's focus on the options available for Windows users, since that's what we are using. we'll see a few different download choices:

OSGeo4W Network Installer:

-  This option is the best if you want to keep QGIS up to date automatically and even run multiple versions of QGIS on your system.
-  The OSGeo4W Installer will download the necessary files as needed, which helps reduce the load on their servers. You can always update QGIS easily using this method.

If you choose this, click the OSGeo4W Network Installer link.

Offline (Standalone) Installers:

If you prefer a simpler installation or want to keep the installer on a USB drive or network share, you can download the Standalone Installer.

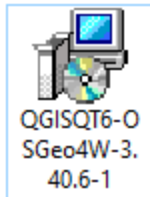
Here are your options:

-  Long Term Version for Windows (3.40 LTR)
-  Latest Version for Windows (3.42)
-  Latest Version for Windows (3.42) with Qt6 (experimental)

Note: Starting from QGIS 3.20, only 64-bit Windows executables are available, so make sure your computer supports that. Go ahead and click on the version you'd like to download. For this tutorial, we'll use the latest version 3.42.

Step 4: Installing QGIS on Windows

Once the download is complete, locate the installer file on your computer (typically in your Downloads folder).

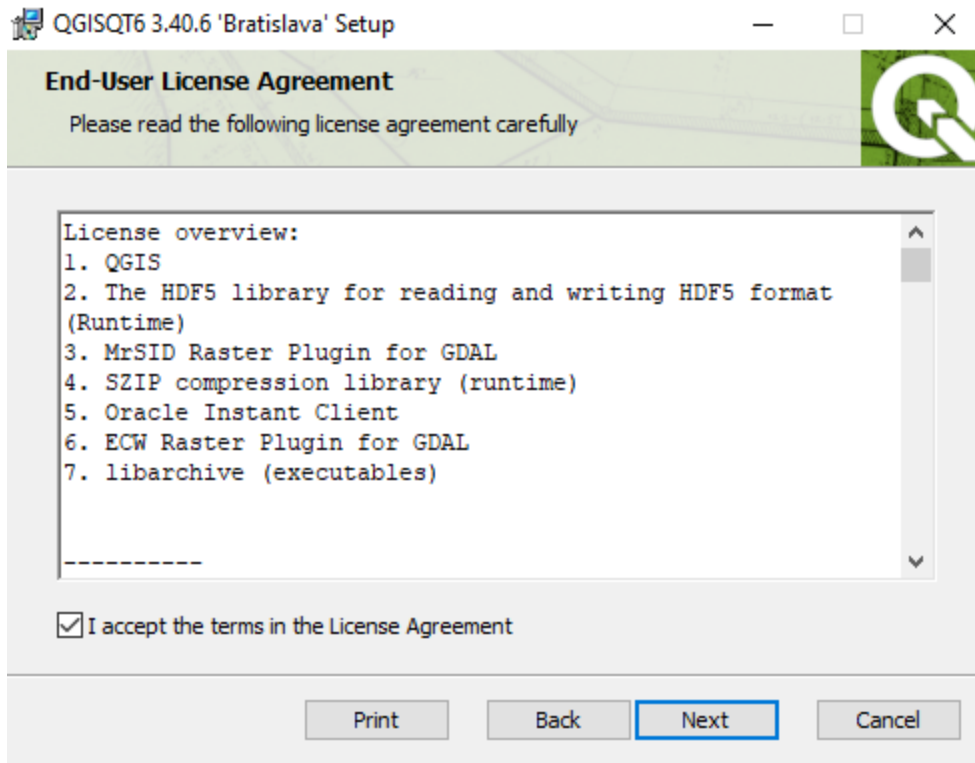


Double-click on the installer to begin the installation process.



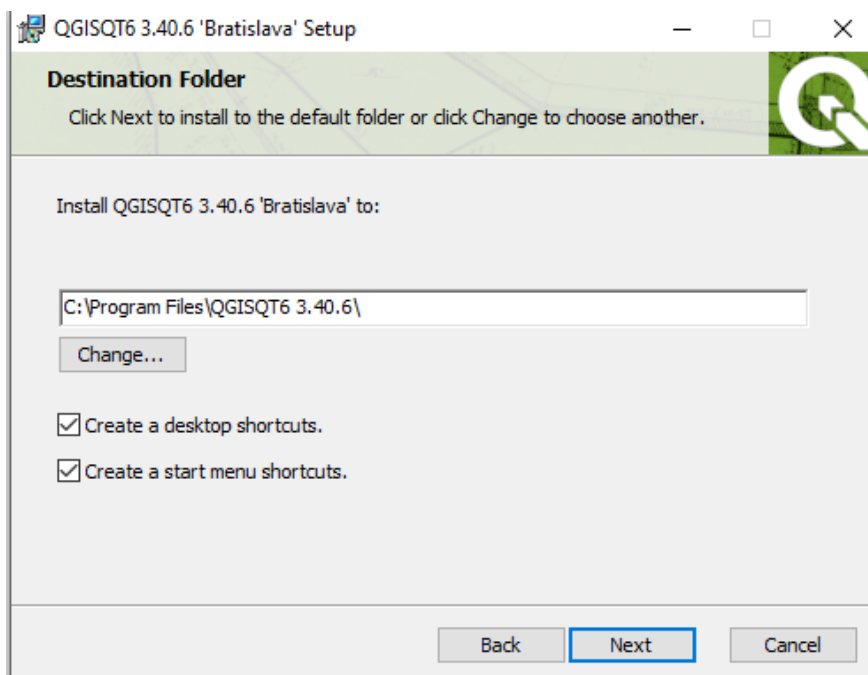
Here's what you need to do next:

- ✚ Choose your language: Pick English (or your preferred language) for the installation.
- ✚ License Agreement: Read through the license agreement and click Accept to proceed.



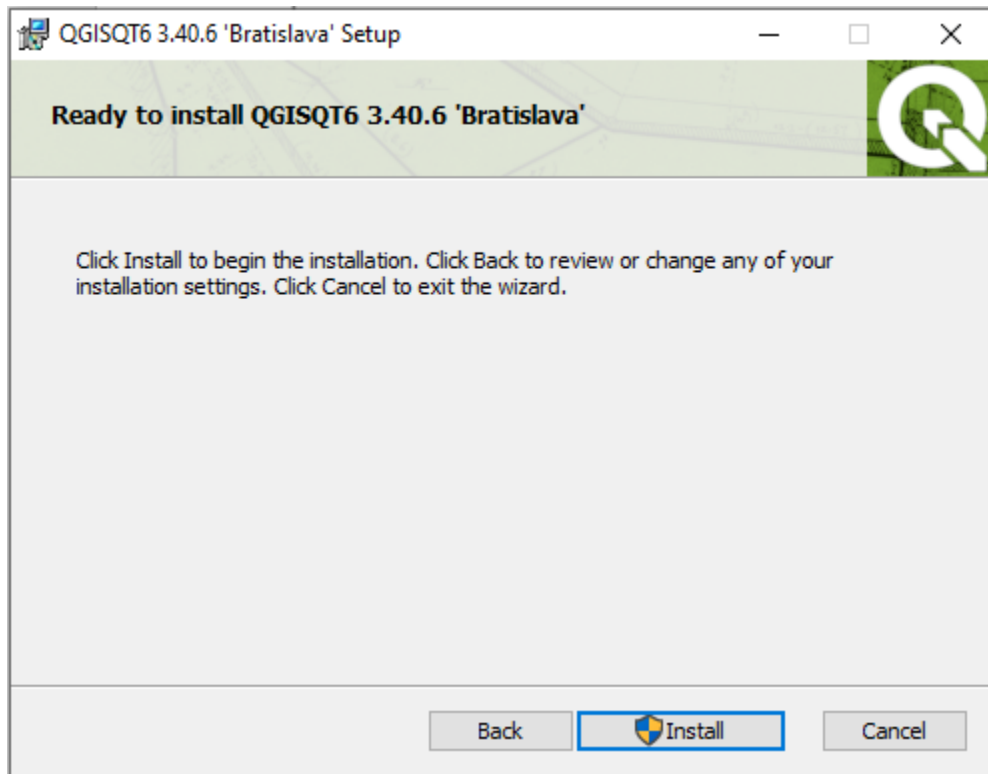
Installation Folder:

- You can use the default installation folder, or choose a different location on your computer.



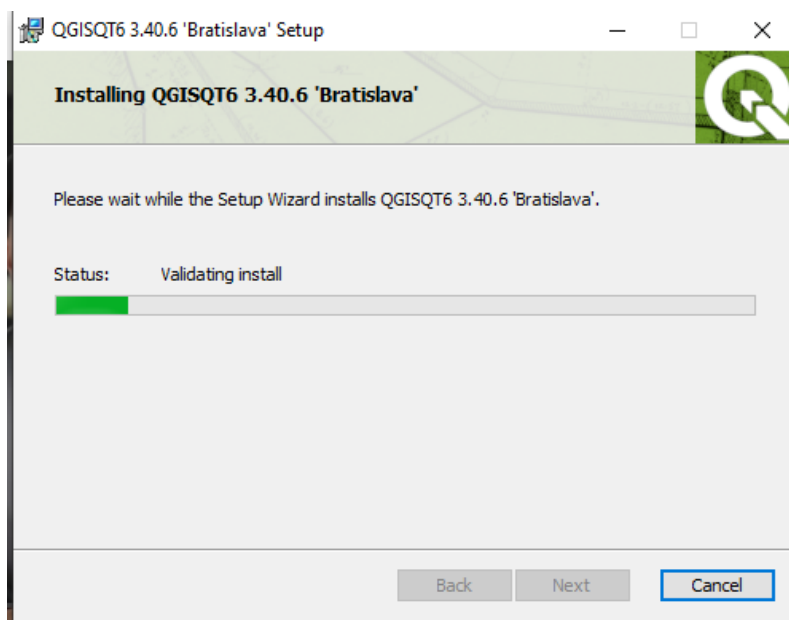
Start the Installation:

Click Install to begin the process.



✚ The installation may take a few minutes, depending on our computer's performance.

Installation Process



Open the installer and follow the prompts to install QGIS.

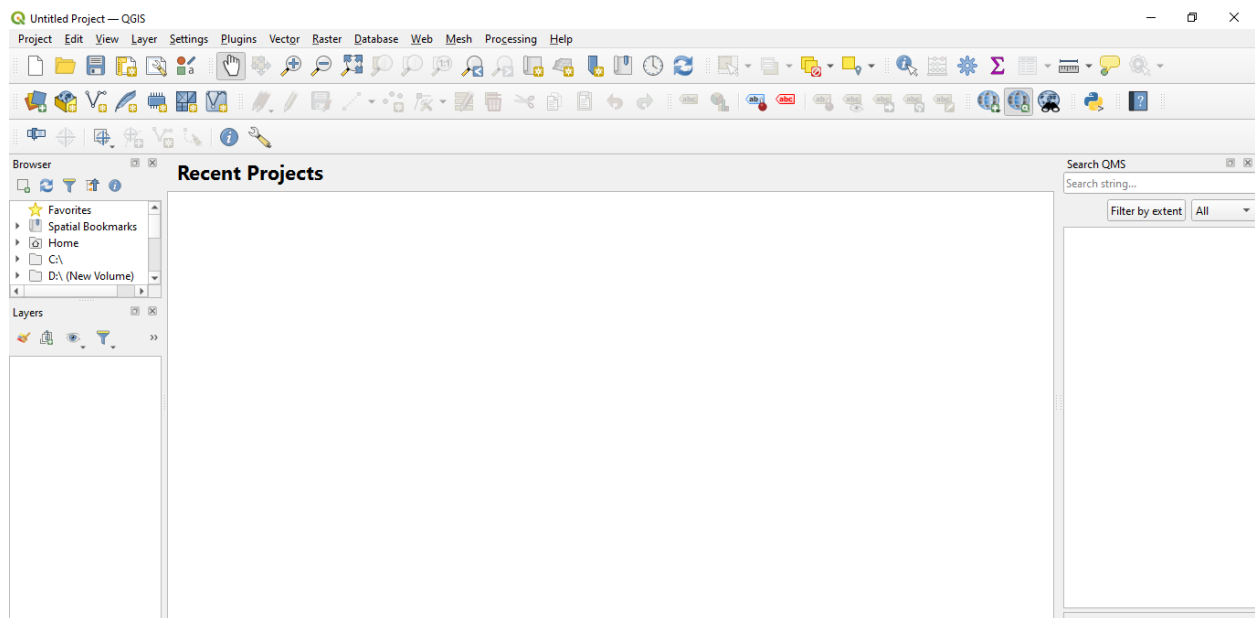
Make sure you select the option to install Python support. This is important for scripting and working with PyQGIS later.

Once the installation is complete, you're ready to launch QGIS for the first time."

Launching QGIS

When we open QGIS, we'll see a range of tools and panels, such as the map canvas, menus, and toolbars.

We'll explore all these features in upcoming videos, but for now, let's set up Python for geospatial analysis.



Setting Up the Python Environment

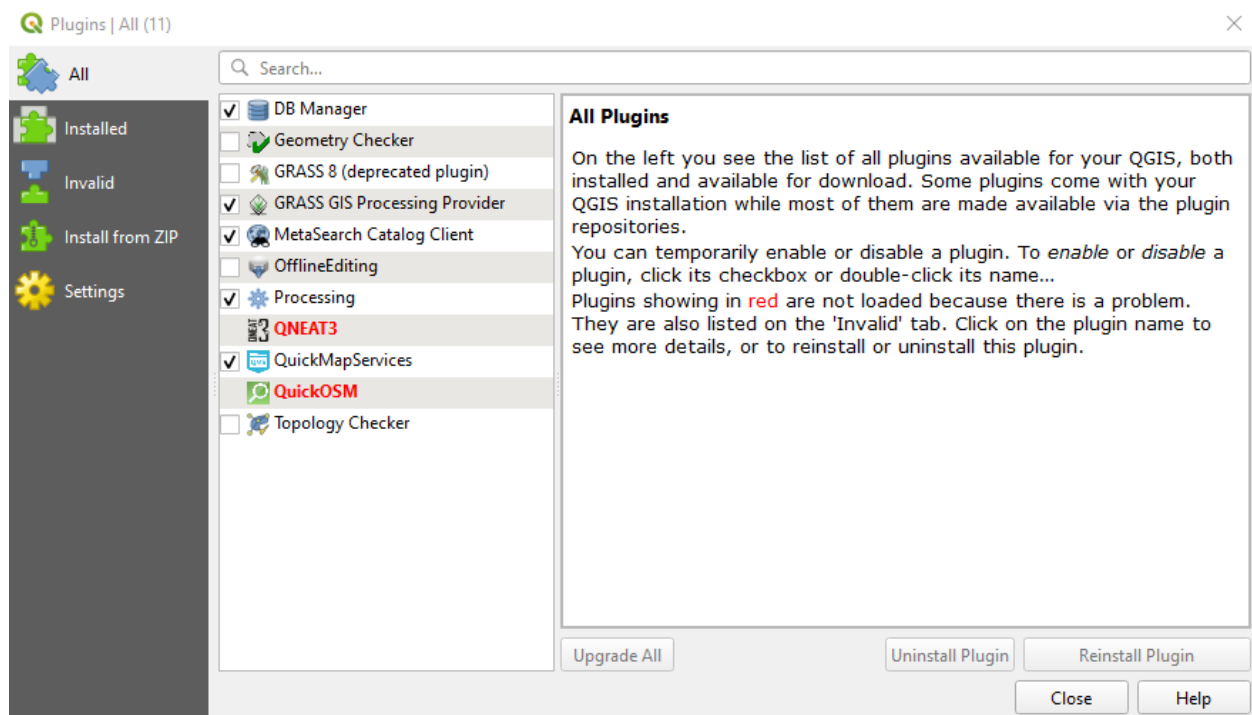
QGIS comes with its own Python interpreter, which means you can start coding immediately.

To access Python, simply go to Plugins > Python Console.

This will open the Python Console where you can run Python commands and interact with QGIS.

Installing Plugins

Go to Plugins > Manage and Install Plugins. Here, you can search for plugins like:



Processing, which provides access to many geospatial algorithms,
QuickMapServices, for adding basemaps,
GeoCoding, and many more. Just select the plugin and click Install to add it to your QGIS setup.

Using PyQGIS

PyQGIS is the Python library that allows you to interact with QGIS.

You can run Python scripts directly in the Python Console or use external editors like VS Code for more complex scripts.

We'll cover PyQGIS scripting but for now, just know that it opens up a world of possibilities for automating tasks and analyzing data in QGIS.

3. Overview of Python Console and Processing Toolbox in QGIS

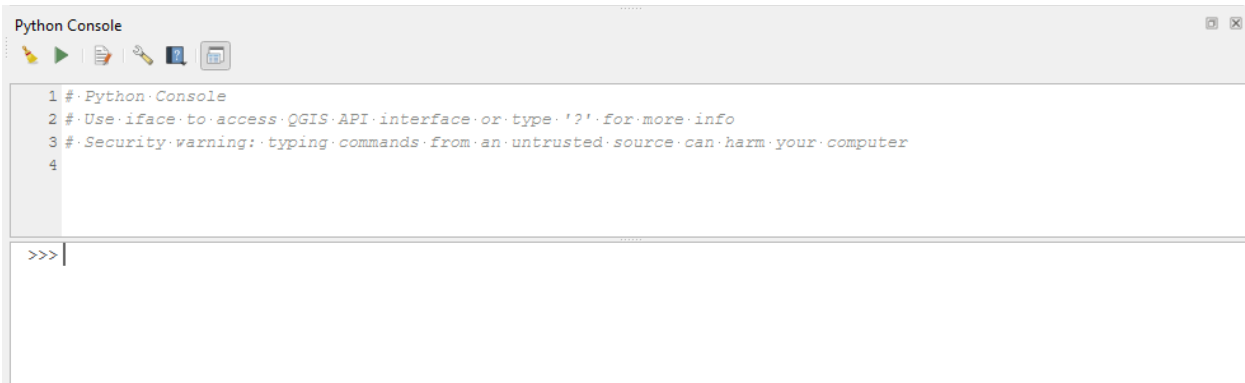
Overview of the Python Console

The Python Console in QGIS is a powerful tool that allows you to interact with QGIS using Python commands.

We can use the console to:

- Execute single lines of code,
- Access QGIS's built-in functions and libraries,
- And automate workflows by writing scripts.

The Python Console is great for experimenting and testing small commands before integrating them into larger scripts.



Overview of the Processing Toolbox

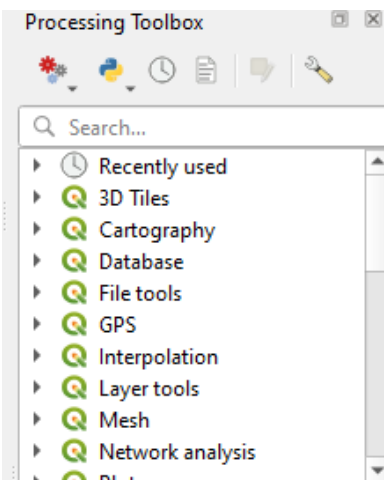
The **Processing Toolbox** is a set of tools that allows you to perform advanced geospatial analysis and automate tasks in QGIS.

We can access it from

- **Processing > Toolbox.**

Inside the toolbox, we'll find a wide range of algorithms for spatial analysis, including buffering, reprojecting, and performing spatial joins.

This toolbox is especially useful when we want to automate workflows or run algorithms on multiple datasets.



Part 2: Basic QGIS Operations Using Python

1. Loading vector and raster layers using Python

Step 1: Open the Python Console

Open QGIS, then go to the Plugins menu and click Python Console. This will open a panel at the bottom where you can enter Python commands directly.

Step 2: Load a Vector and Raster Layer

To load a shape file, we use the following Python code:

```
1  # Import required QGIS module
2  from qgis.core import QgsVectorLayer, QgsProject
3
4  # Define the path to the vector file
5  vector_file = "C:/path/to/your/file.shp" # Change this to your file path
6
7  # Define a name for the layer (as it will appear in QGIS)
8  layer_name = "My Vector Layer"
9
10 # Create the vector layer
11 vector_layer = QgsVectorLayer(vector_file, layer_name, "ogr")
12
13 # Check if the layer is valid
14 ✓ if not vector_layer.isValid():
15     print("Layer failed to load!")
16 ✓ else:
17     # Add the layer to the QGIS project
18     QgsProject.instance().addMapLayer(vector_layer)
19     print("Layer loaded successfully!")
20
```

`Qgis.core` is a **Python module** that provides access to the core functionality of **QGIS (Quantum GIS)** through Python. It is part of the **PyQGIS API**, which allows users to interact with QGIS from scripts and plugins.

What `qgis.core` Contains

It includes classes and functions related to:

- Vector and raster data handling
- Layer management (e.g., `QgsVectorLayer`, `QgsRasterLayer`)
- Geometry and feature operations (e.g., `QgsFeature`, `QgsGeometry`)
- Coordinate reference systems (e.g., `QgsCoordinateReferenceSystem`)
- Map rendering and project management (e.g., `QgsProject`)
- Spatial analysis tools

When to Use `qgis.core`

Use `qgis.core` when:

- Automating GIS workflows using Python.
- Developing QGIS plugins.
- Creating standalone Python scripts that need to interact with GIS data.

Note

To use `qgis.core`, you need to have QGIS installed and your Python environment must be properly set up to use its libraries (i.e., use the QGIS-provided Python or set up the environment variables manually).

`QgsVectorLayer` is a **class in the `qgis.core` module** that represents a **vector data layer** in QGIS (like a shapefile, GeoJSON, or database layer).

What `QgsVectorLayer` Does

The `QgsVectorLayer` class provides:

- Methods to **load**, **read**, and **manipulate** vector data
- Access to features and attributes
- Editing and saving capabilities
- Integration with map rendering and analysis

`QgsProject` is a class in the `qgis.core` module that represents the **entire QGIS project** in memory — similar to a `.qgz` or `.qgs` file you work with in QGIS Desktop.

What Is `QgsProject` Used For?

It is used to:

- ◆ Manage all layers loaded into the project
- ◆ Add, remove, and access map layers
- ◆ Save or load QGIS project files (.qgs or .qgz)
- ◆ Manage project-level settings (CRS, metadata, variables, etc.)

When to Use `QgsProject`

Use `QgsProject` when:

- You're scripting a QGIS session and want to manage layers programmatically
- You're writing a plugin or automation that mimics user interaction in QGIS
- You want to **save** or **load** a full QGIS project structure

Example 1. If our shapefile is available in the folder

- ✓ *D:\QGIS Tutorials\ne_10m_admin_0_countries*
- ✓ *And the file name is ne_10m_admin_0_countries.shp,*
- ✓ *Then this exercise will show you how to load that shapefile into QGIS using Python.*

Step1. Import Required QGIS Modules

```
from qgis.core import QgsVectorLayer, QgsProject
```

- ✓ **from** – Python keyword for importing specific items from a module.
- ✓ **qgis.core** – **module**: QGIS core library.
- ✓ **QgsVectorLayer, QgsProject** – **classes**: Used to handle vector layers and the QGIS project.

This command loads the necessary tools to work with vector layers and manage them in the QGIS project.

Step 2: Define the File Path to Your Shapefile

Now we tell QGIS where our shapefile is stored.

```
shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
```

- ✓ **vector_path** – **variable** storing the file path to the shapefile.
- ✓ **String** value: the file path.

Note the r before the string. It means raw string this helps Python treat the backslashes (\) correctly.

Step 3: Give Your Layer a Name

This is the name that will appear in the QGIS Layers Panel:

```
layer_name = "World Countries"
```

Step 4: Create the Vector Layer

Now we use the file path and name to create a layer in Python:

```
vector_layer = QgsVectorLayer(shapefile_path, layer_name, "ogr")
```

- `shapefile_path` is the full path to your `.shp` file
- `layer_name` is the name that will appear in QGIS
- `"ogr"` is the data provider used for shapefiles and many other vector formats

Step 5: Check if the Layer Was Loaded Successfully

```
if vector_layer.isValid():
```

- ✓ **if** – Python conditional.
- ✓ **isValid()** – **method**: checks if the shape file was successfully loaded.

```
print("Layer loaded successfully!")
```

- ✓ **print()** – **built-in function** to display a message.

```
QgsProject.instance().addMapLayer(vector_layer)
```

- ✓ **QgsProject.instance()** – **static method** to access the current project.
- ✓ **addMapLayer(...)** – **method** to add the layer to the QGIS map view.

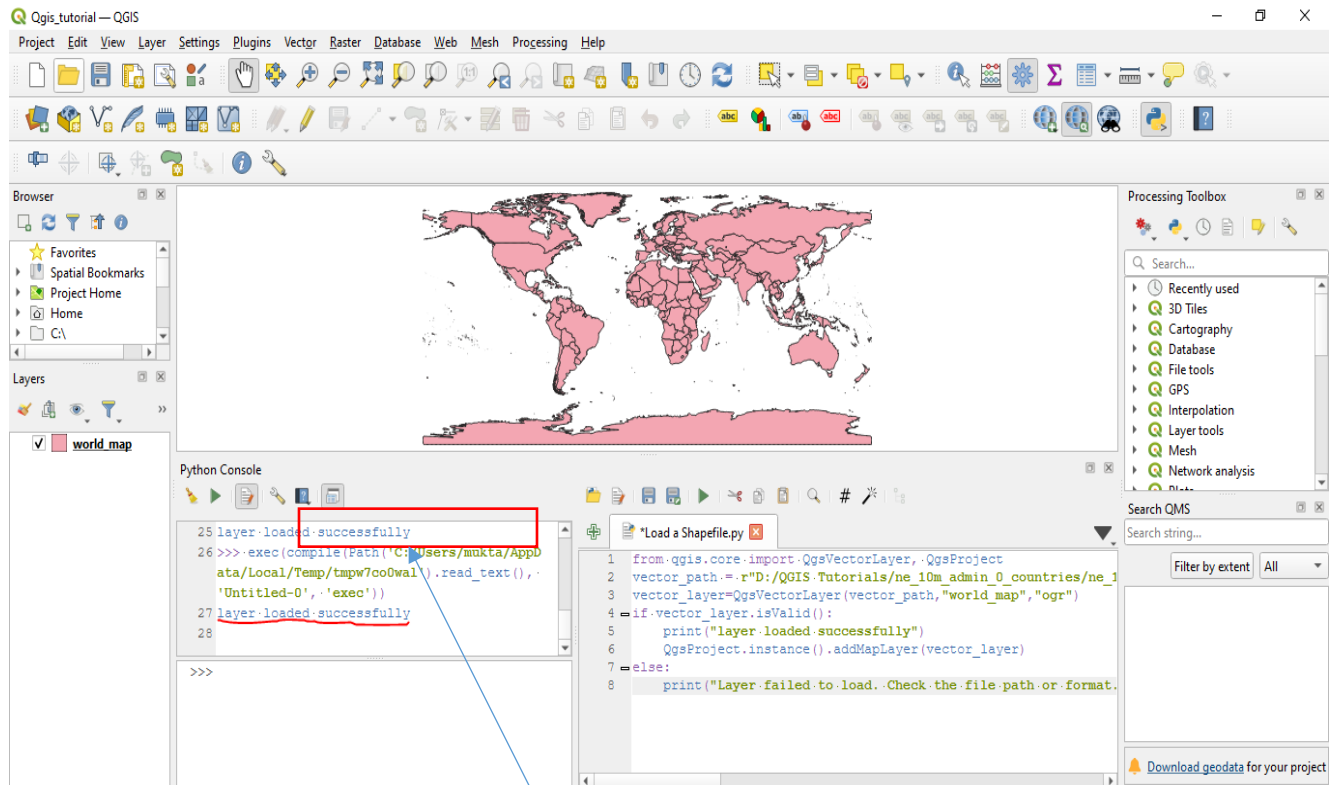
```
else:
```

```
print("Failed to load the layer.")
```

- ✓ **else** – Python keyword for alternate path.
- ✓ Another **print ()** for error reporting.

If everything was correct:

- Your layer will now appear in the **QGIS Layers Panel**
- You'll see this message: Layer loaded successfully into QGIS!



Layer loaded successfully

To load a Raster Data we use the following Python code:

```
1  from qgis.core import QgsRasterLayer, QgsProject
2
3  # Replace this with the full path to your raster file
4  raster_file_path = r"FULL\PATH\TO\YOUR\RASTER.tif"
5
6  # Set a name that will appear in the QGIS Layers Panel
7  layer_name = "My Raster Layer"
8
9  # Create the raster layer
10 raster_layer = QgsRasterLayer(raster_file_path, layer_name)
11
12 # Check if it loaded correctly
13 v if not raster_layer.isValid():
14     print("❌ Failed to load raster layer. Check the file path and format.")
15 v else:
16     # Add the raster layer to the
17
```

- Replace `FULL\PATH\TO\YOUR\RASTER.tif` with your actual raster file path.
- You can use `.tif`, `.img`, `.asc`, etc., as long as QGIS supports the format.

`QgsRasterLayer` is a class in the `qgis.core` module that allows us to **load, manage, and analyze raster data** (like satellite images, digital elevation models, aerial photos, etc.) in QGIS using Python.

What `QgsRasterLayer` Does

It provides tools to:

- Load raster datasets
- Access metadata (band count, resolution, CRS, etc.)
- Perform operations like band extraction, resampling, rendering, etc.
- Add the layer to the QGIS map/project

When to Use `QgsRasterLayer`

Use it when you want to:

- Load and visualize satellite images or DEMs in a script
- Perform remote sensing analysis

- Access raster statistics or pixel-level information
- Integrate raster processing in your plugin or automation

Example 2. If your raster file is available in the folder:

- ✓ **D:\QGIS Tutorials\dem_data**
- ✓ **And the file name is dem.tif,**
- ✓ **Then this exercise will show you how to load that raster file into QGIS using Python.**

Step1: Import Required QGIS Classes

In the console, type:

```
from qgis.core import QgsRasterLayer, QgsProject
```

This will let us create raster layers and add them to the map.

Step2. Define the File Path to the Raster File

```
raster_file = r"D:\QGIS Tutorials\dem_data\dem.tif"
```

The r before the string is a raw string format to handle Windows paths safely.

Step 3: Set a Layer Name

```
layer_name = "Elevation Model"
```

This is the name that will show up in QGIS.

Step 4: Create the Raster Layer

```
raster_layer = QgsRasterLayer(raster_file, layer_name)
```

- `raster_file` is the path to your `.tif` file
- `layer_name` is the display name

Step 5: Check if the Layer Was Loaded Successfully

```

1 ✓ if raster_layer.isValid():
2     QgsProject.instance().addMapLayer(raster_layer)
3     print(" Raster layer loaded successfully into QGIS!")
4 ✓ else:
5     print(" Raster layer failed to load. Check the path or file.")
6

```

- ✓ .isValid() is a **method** of the QgsRasterLayer class.
- ✓ It checks if the raster file was loaded correctly.
- ✓ The if statement is a **control structure** to determine what happens next.

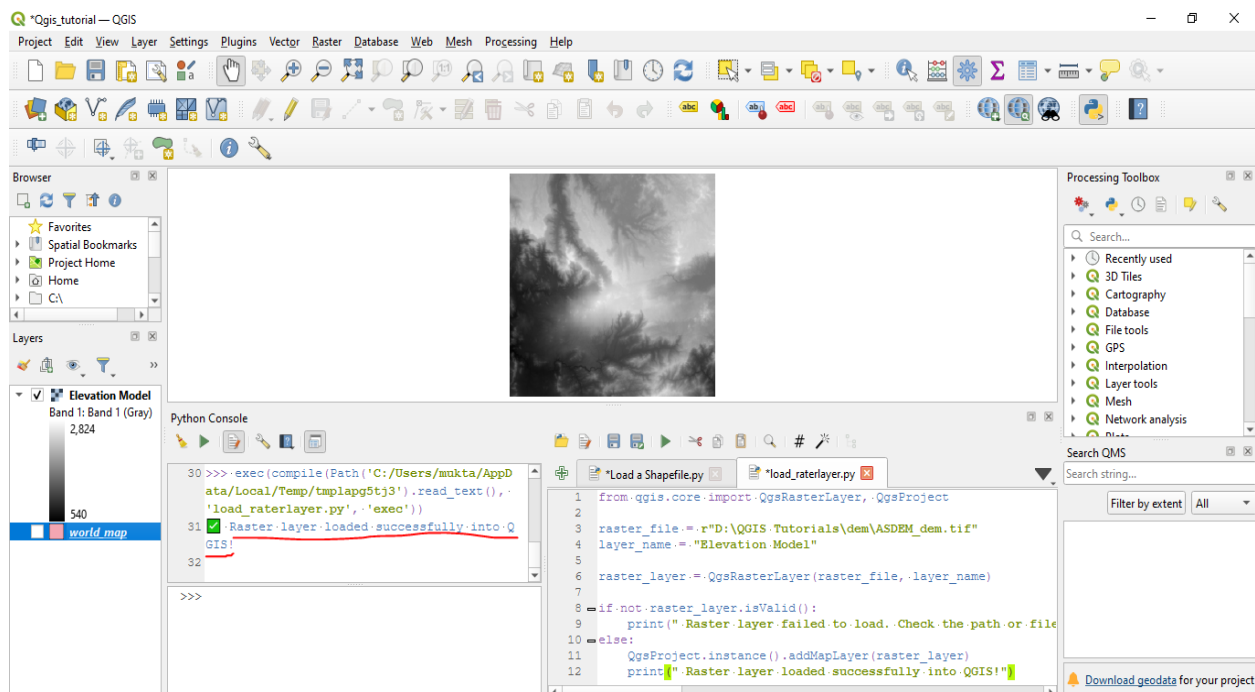
`QgsProject.instance().addMapLayer(raster_layer)`

`QgsProject.instance ()` is a class method that returns the current QGIS project instance.

`.addMapLayer ()` is a method that adds the given layer to the project.

This line inserts your raster into the QGIS interface for visualization.

`print(" Raster layer loaded successfully into QGIS!")`



2. Layer styling and attribute manipulation

Styling layers and modifying attributes are among the most essential parts of spatial data visualization and analysis. In QGIS, the PyQGIS API provides powerful tools to customize how vector layers appear on the map and to interact with their attribute data programmatically.

In this example, we will:

- Load an administrative boundary shapefile (ETH_adm3.shp)
- Change its style (color, outline, transparency)
- Modify or access attribute values
- Apply a filter to show only specific records (e.g., by region or district)

This is important for:

- Thematic mapping (e.g., showing districts by population or category)
- Automating visual styles for consistent reports
- Data cleaning and preprocessing

Let's walk through it step-by-step.

Requirements

Make sure the following conditions are met before running the code:

File path: D:\QGIS Tutorials\ETH_adm\ETH_adm3.shp

QGIS is installed and the Python Console (or external PyQGIS environment) is ready.

```

1  from qgis.core import (
2      QgsVectorLayer,
3      QgsProject,
4      QgsFillSymbol,
5      QgsCategorizedSymbolRenderer,
6      QgsRendererCategory,
7      QgsField
8  )
9  from qgis.PyQt.QtCore import QVariant
10
11  # Step 1: Load the vector layer
12  shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
13  layer_name = "World Countries"
14  vector_layer = QgsVectorLayer(shapefile_path, layer_name, "ogr")
15
16  if not vector_layer.isValid():
17      print("✗ Layer failed to load. Check the path.")
18  else:
19      QgsProject.instance().addMapLayer(vector_layer)
20      print("✓ Layer loaded successfully!")
21
22  # Step 2: Apply categorized styling based on continent or region
23  field_name = "CONTINENT" # You can also try "REGION_UN" or "SUBREGION"
24
25  # Define colors for each continent
26  continent_colors = {
27      "Africa": "orange",
28      "Asia": "red",
29      "Europe": "blue",
30      "North America": "green",
31      "South America": "purple",
32      "Oceania": "yellow",
33      "Seven seas (open ocean)": "gray"
34  }
35
36  categories = []
37
38  for continent, color in continent_colors.items():
39      symbol = QgsFillSymbol.createSimple({
40          'color': color,
41          'outline_color': 'black',
42          'outline_width': '0.2'
43      })
44      category = QgsRendererCategory(continent, symbol, continent)
45      categories.append(category)
46
47  ...

```

```

47     renderer = QgsCategorizedSymbolRenderer(field_name, categories)
48     vector_layer.setRenderer(renderer)
49     vector_layer.triggerRepaint()
50
51     print("👉 Styling applied based on the 'CONTINENT' field.")
52
53     # Step 3: Print country names (optional)
54     print("🌐 List of country names:")
55     for feature in vector_layer.getFeatures():
56         print("-", feature["NAME"])
57
58     # Step 4: Add and populate a new field 'AreaNote'
59     vector_layer.startEditing()
60     vector_layer.dataProvider().addAttributes([QgsField("AreaNote", QVariant.String)])
61     vector_layer.updateFields()
62
63     area_field_index = vector_layer.fields().indexOfName("AreaNote")
64
65     for feature in vector_layer.getFeatures():
66         area_km2 = feature.geometry().area() / 1e6 # approx. in sq. km
67         note = "Large" if area_km2 > 1000000 else "Small"
68         vector_layer.changeAttributeValue(feature.id(), area_field_index, note)
69
70     vector_layer.commitChanges()
71     print("📝 'AreaNote' field added and filled based on country area.")
72

```

`QgsFillSymbol` is a class in the `qgis.core` module that defines **how polygon features are visually styled or filled** in QGIS using Python.

What is `QgsFillSymbol`?

It is used to **create and manage the fill style** of **polygon layers**, such as:

- Land cover zones
- Administrative boundaries
- Lakes, parks, parcels

When to Use `QgsFillSymbol`

Use it when you:

- Style polygon layers dynamically using Python
- Automate map styling
- Create custom styles for thematic maps
- Develop plugins that control map rendering

Explanation of Python Concepts Used

Import necessary QGIS and Qt modules

```
from qgis.core import (  
    QgsVectorLayer,  
    QgsProject,  
    QgsFillSymbol,  
    QgsCategorizedSymbolRenderer,  
    QgsRendererCategory,  
    QgsField  
)  
from qgis.PyQt.QtCore import QVariant
```

`QgsCategorizedSymbolRenderer` is a class in the `qgis.core` module used to apply **category-based styling** (thematic mapping) to **vector layers**, especially **polygons and lines** in QGIS using Python.

Use Cases

1. Style Based on Land Cover Type

Example: You have a polygon layer showing different land types like:

- Forest
- Urban
- Agricultural
- Water

You can use `QgsCategorizedSymbolRenderer` to **assign a different color to each land type** based on a field called "landcover".

Why?

So users can easily distinguish between land types by color in your map.

2. Differentiate by Region, Zone, Status, Class, or Group

Example: You're mapping districts or administrative zones, and there's a field "zone" with values like:

- North
- South
- East
- West

You can categorize the map based on this field so **each zone has a distinct style**.

Other examples:

- "status": active / inactive
- "risk_class": high / medium / low
- "ownership": public / private / community

Why?

This makes **data interpretation faster and visually intuitive** in thematic maps.

3. Automate Thematic Mapping Using Scripts

 **Example:** You are developing a script or plugin that:

- Loads vector layers
- Styles them based on their attribute values
- Outputs styled maps without user interaction

Using `QgsCategorizedSymbolRenderer`, you can **automate map styling** for many layers, which is especially useful in:

- Batch processing
- Reporting workflows
- Web map generation

Why?

It saves time and reduces human error by **automating style creation** for categories.

`QgsRendererCategory` is a **building block** — it represents a **single category** for styling features based on an attribute value.

□ It stores:

- A **value** (e.g., "Residential")
- A **symbol** (e.g., red fill for residential zones)
- A **label** (what to display in the legend, e.g., "Residential Area")

The Difference between `QgsRendererCategory` and `QgsCategorizedSymbolRenderer` is explain in the table below

Feature	<code>QgsRendererCategory</code>	<code>QgsCategorizedSymbolRenderer</code>
Represents	One category/value/style rule	The entire renderer that applies multiple categories
What it stores	A single value , symbol, and label	A field name + list of categories
Use	You define many of these for each value	You give these to the renderer to apply them to the layer
Analogy	A recipe card	The cookbook that uses all the cards to cook a full meal

`QgsField` is a class in the `qgis.core` module that represents a **single attribute field** (i.e. a column) in a vector layers attribute table.

Common Uses

- Creating new fields when adding attributes to layers
- Reading metadata about fields
- Defining fields in memory layers or shapefile outputs

Meaning:

- These lines import core classes needed to work with vector layers, symbology, rendering, and field editing.
- `QVariant` is used to define the **data type** of new attribute fields (like string, number, etc.).

Usage:

- You need these classes for loading shapefiles, changing layer styles, and editing attribute tables.

Load the vector layer

```
shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
layer_name = "World Countries"
vector_layer = QgsVectorLayer(shapefile_path, layer_name, "ogr")
```

✓ Meaning:

- `shapefile_path` : Path to your `.shp` file.
- `layer_name` : The name that will appear in QGIS.
- `QgsVectorLayer(...)` : Loads the shapefile using the OGR provider (a standard for shapefiles).

✓ Usage:

- This loads the shapefile into QGIS memory so you can work with it in the Python console.

Check if layer is valid and add to QGIS

```
if not vector_layer.isValid():
    print("✗ Layer failed to load. Check the path.")
else:
    QgsProject.instance().addMapLayer(vector_layer)
    print("✓ Layer loaded successfully!")
```

✓ Meaning:

- `.isValid()` checks if the file was loaded correctly.
- `QgsProject.instance().addMapLayer(...)` adds the layer to the active QGIS project.

✓ Usage:

- Always check validity to avoid silent errors.
- This step is necessary to **display the layer on the QGIS canvas**.

Apply categorized symbology (style by continent)

```
field_name = "CONTINENT"
```

Meaning:

Choose the field to base your categorization on (e.g., "CONTINENT").

```
continent_colors = {  
    "Africa": "orange",  
    "Asia": "red",  
    "Europe": "blue",  
    "North America": "green",  
    "South America": "purple",  
    "Oceania": "yellow",  
    "Seven seas (open ocean)": "gray"  
}
```

✓ Meaning:

- A dictionary assigning a unique color to each continent.

✓ Usage:

- These colors will be used for styling features in the next steps.

Create categorized symbols

```
categories = []

for continent, color in continent_colors.items():
    symbol = QgsFillSymbol.createSimple({
        'color': color,
        'outline_color': 'black',
        'outline_width': '0.2'
    })
    category = QgsRendererCategory(continent, symbol, continent)
    categories.append(category)
```

✓ Meaning:

- Loop through each continent and create a unique **fill symbol** (colored interior with black border).
- `QgsRendererCategory(...)` defines a rule: "If CONTINENT == 'Africa', use orange symbol."

✓ Usage:

- Used to **visually differentiate continents** with unique styles.

Apply the categorized renderer to the layer

```
renderer = QgsCategorizedSymbolRenderer(field_name, categories)
vector_layer.setRenderer(renderer)
vector_layer.triggerRepaint()
```

✓ Meaning:

- `QgsCategorizedSymbolRenderer` : Applies different symbols based on field values.
- `setRenderer` : Sets this renderer for the current vector layer.
- `triggerRepaint` : Refreshes the map view with new styles.

✓ Usage:

- This updates the **visual representation** of the layer based on the selected field.

Print all country names

```
print("🌍 List of country names:")  
for feature in vector_layer.getFeatures():  
    print(" -", feature["NAME"])
```

✓✓ Meaning:

Iterates over each feature (country) and prints the value in the "NAME" field.

✓✓ Usage:

This is a good way to inspect your attribute data and verify fields.

Start editing the layer

```
vector_layer.startEditing()
```

✓✓ Meaning:

Puts the layer in edit mode, so we can add or modify attributes.

✓✓ Usage:

Required before changing or adding data to a layer.

Add a new field (AreaNote)

```
vector_layer.dataProvider().addAttributes([QgsField("AreaNote", QVariant.String)])  
vector_layer.updateFields()
```

✔ Meaning:

Adds a new column named "AreaNote" with string data type.

updateFields() refreshes the layer's field list after the change.

✔ Usage:

Allows us to store custom values like "Large" or "Small" for each country.

Populate the new field with conditional values

```
area_field_index = vector_layer.fields().indexOfName("AreaNote")

for feature in vector_layer.getFeatures():
    area_km2 = feature.geometry().area() / 1e6 # approx. sq. km
    note = "Large" if area_km2 > 1000000 else "Small"
    vector_layer.changeAttributeValue(feature.id(), area_field_index, note)
```

✔ Meaning:

- Calculate each country's area (in sq. km)
- If area > 1 million, set AreaNote = "Large"; else "Small"
- changeAttributeValue(...) updates the field for each feature.

✔ Usage:

- Demonstrates **attribute manipulation based on geometry**.

Commit (save) the changes

```
vector_layer.commitChanges()
print("📝 'AreaNote' field added and filled based on country area.")
```

✔ Meaning:

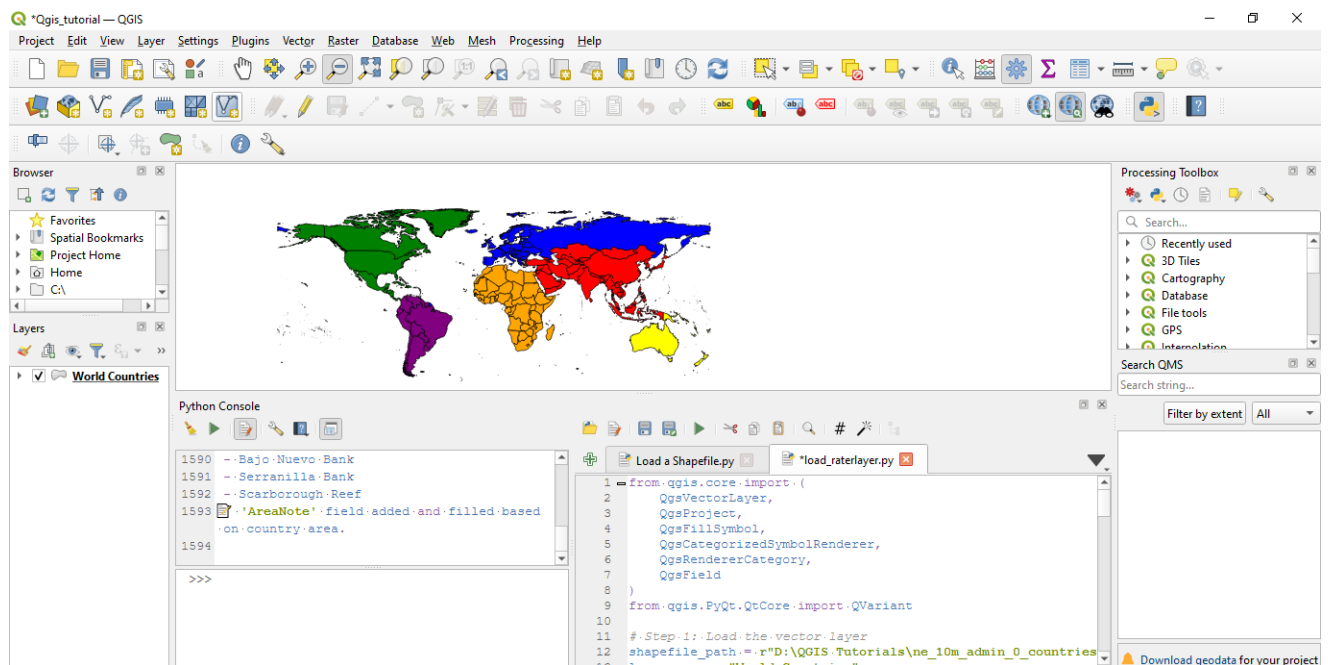
Saves all edits to the layer permanently.

The new AreaNote values will now appear in the attribute table.

✔ Usage:

Required to finalize your edits.

Code Section	Purpose
<code>QgsVectorLayer</code>	Load shapefile
<code>QgsProject.addMapLayer()</code>	Add layer to the project
<code>QgsFillSymbol</code> , <code>Renderer</code>	Style layer by color
<code>getFeatures()</code> + <code>"NAME"</code>	Read attribute values (like country names)
<code>startEditing()</code> → <code>commit()</code>	Add new field and update values
<code>geometry().area()</code>	Use spatial (geometric) data



3. Automating map creation with Python

Creating maps manually in QGIS is great for one-time projects, but when you need to produce many maps especially with different extents, layers, or attribute-driven titles automation

becomes essential. PyQGIS provides powerful classes like QgsLayout, QgsLayoutItemMap, and QgsLayoutExporter that allow you to build, style, and export maps directly from Python.

In this section, we will:

- Load a vector or raster layer
- Create a print layout automatically
- Add a map canvas and a title
- Export the map as a PNG or PDF file

This technique is useful for:

- Generating maps for each district or zone
- Batch export of thematic maps
- Building dynamic reports with location-specific content

```
1  from qgis.core import (
2      QgsProject,
3      QgsPrintLayout,
4      QgsLayoutItemMap,
5      QgsLayoutItemLabel,
6      QgsLayoutItemScaleBar,
7      QgsLayoutItemLegend,
8      QgsLayoutItemPicture,
9      QgsUnitTypes,
10     QgsLayoutExporter,
11     QgsLayoutPoint,
12     QgsLayoutSize
13 )
14 from qgis.PyQt.QtGui import QFont
15 from qgis.PyQt.QtCore import QSizeF
16
17 # Step 1: Prepare layout
18 project = QgsProject.instance()
19 layout_manager = project.layoutManager()
20
21 layout = QgsPrintLayout(project)
22 layout.initializeDefaults()
23 layout.setName("World Map Layout")
24
```

```

25     # Remove existing layout with same name
26     for l in layout_manager.printLayouts():
27         if l.name() == "World Map Layout":
28             layout_manager.removeLayout(l)
29
30     layout_manager.addLayout(layout)
31
32     # Step 2: Map Item
33     map_item = QgsLayoutItemMap(layout)
34     map_item.setRect(20, 20, 180, 120)
35
36     layer = project.mapLayersByName("World Countries")[0]
37     map_item.setExtent(layer.extent())
38     layout.addLayoutItem(map_item)
39
40     # Step 3: Title
41     title = QgsLayoutItemLabel(layout)
42     title.setText("World Countries Map")
43     title.setFont(QFont("Arial", 18))
44     title.adjustSizeToText()
45     title.attemptMove(QgsLayoutPoint(20, 5))
46     layout.addLayoutItem(title)
47
48     # Step 4: Scale Bar
49     scale_bar = QgsLayoutItemScaleBar(layout)
50     scale_bar.setStyle('Single Box')
51     scale_bar.setLinkedMap(map_item)
52     scale_bar.applyDefaultSize()
53     scale_bar.setNumberOfSegments(4)
54     scale_bar.setUnits(QgsUnitTypes.DistanceKilometers)
55     scale_bar.setUnitLabel("km")
56     scale_bar.setFont(QFont("Arial", 10))
57     scale_bar.attemptMove(QgsLayoutPoint(20, 150))
58     layout.addLayoutItem(scale_bar)
59
60     # Step 5: Legend
61     legend = QgsLayoutItemLegend(layout)
62     legend.setTitle("Legend")
63     legend.setLinkedMap(map_item)
64     legend.updateLegend()
65     legend.attemptMove(QgsLayoutPoint(170, 20)) # right side
66     layout.addLayoutItem(legend)
67     --

```

```

67
68 # Step 6: North Arrow (Picture)
69 north_arrow = QgsLayoutItemPicture(layout)
70 north_arrow.setPicturePath(QgsApplication.defaultThemePath() + "/north_arrows/layout_north_arrow_01.svg")
71 north_arrow.setFixedSize(QgsLayoutSize(20, 20)) # in mm
72 north_arrow.attemptMove(QgsLayoutPoint(170, 150))
73 layout.addLayoutItem(north_arrow)
74
75 # Step 7: CRS Label (coordinate system info)
76 crs_label = QgsLayoutItemLabel(layout)
77 crs_text = "CRS: " + layer.crs().authid() # e.g. EPSG:4326
78 crs_label.setText(crs_text)
79 crs_label.setFont(QFont("Arial", 9))
80 crs_label.adjustSizeToText()
81 crs_label.attemptMove(QgsLayoutPoint(20, 165))
82 layout.addLayoutItem(crs_label)
83
84 # Step 8: Export Layout
85 exporter = QgsLayoutExporter(layout)
86 output_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\world_map_complete.png"
87 result = exporter.exportToImage(output_path, QgsLayoutExporter.ImageExportSettings())
88
89 # Step 9: Feedback
90 if result == QgsLayoutExporter.Success:
91     print("✅ Map with all elements exported successfully to:", output_path)
92 else:
93     print("❌ Export failed.")
94

```

Explanation of the Workflow

A. Import Required Modules

```

from qgis.core import (
    QgsProject, QgsPrintLayout, QgsLayoutItemMap, QgsLayoutItemLabel,
    QgsLayoutItemScaleBar, QgsLayoutItemLegend, QgsLayoutItemPicture,
    QgsUnitTypes, QgsLayoutExporter, QgsLayoutPoint, QgsLayoutSize
)
from qgis.PyQt.QtGui import QFont
from qgis.PyQt.QtCore import QSizeF

```

These modules let us access QGIS layout tools and PyQt GUI components like font and sizing.

- `QgsLayoutPoint` is used for positioning layout elements.
- `QgsLayoutSize` is used to size elements like the north arrow.

B. Access the QGIS Project and Create Layout

```
project = QgsProject.instance()
layout_manager = project.layoutManager()
layout = QgsPrintLayout(project)
layout.initializeDefaults()
layout.setName("World Map Layout")
```

- Gets the current QGIS project.
- Accesses the layout manager to create and manage map layouts.
- Initializes a new layout with default page size and settings.

C. Remove Layout with Duplicate Name (If Exists)

```
for l in layout_manager.printLayouts():
    if l.name() == "World Map Layout":
        layout_manager.removeLayout(l)
layout_manager.addLayout(layout)
```

- Avoids duplicate layouts by removing old versions with the same name.

D. Add the Map Item

```
map_item = QgsLayoutItemMap(layout)
map_item.setRect(20, 20, 180, 120)
layer = project.mapLayersByName("World Countries")[0]
map_item.setExtent(layer.extent())
layout.addLayoutItem(map_item)
```

- Creates the main map frame.
- setExtent() fits the map to the extent (boundary) of your vector layer.
- Assumes your loaded layer is named "World Countries".

E. Add a Title

```

title = QgsLayoutItemLabel(layout)
title.setText("World Countries Map")
title.setFont(QFont("Arial", 18))
title.adjustSizeToText()
title.attemptMove(QgsLayoutPoint(20, 5))
layout.addLayoutItem(title)

```

- Adds a title label to the layout.
- QFont("Arial", 18) sets font size.
- attemptMove() places it at a fixed point (in mm) from the top-left corner.

F. Add a Scale Bar

```

scale_bar = QgsLayoutItemScaleBar(layout)
scale_bar.setStyle('Single Box')
scale_bar.setLinkedMap(map_item)
scale_bar.applyDefaultSize()
scale_bar.setNumberOfSegments(4)
scale_bar.setUnits(QgsUnitTypes.DistanceKilometers)
scale_bar.setUnitLabel("km")
scale_bar.setFont(QFont("Arial", 10))
scale_bar.attemptMove(QgsLayoutPoint(20, 150))
layout.addLayoutItem(scale_bar)

```

- Shows map scale in kilometers.
- Must be linked to your map item.
- Placed near the bottom-left corner of the map.

G. Add a Legend

```

legend = QgsLayoutItemLegend(layout)
legend.setTitle("Legend")
legend.setLinkedMap(map_item)
legend.updateLegend()
legend.attemptMove(QgsLayoutPoint(170, 20))
layout.addLayoutItem(legend)

```

- Automatically includes all visible layers on your map.
- Linked to your map and placed on the right-hand side.

H. Add a North Arrow

```
north_arrow = QgsLayoutItemPicture(layout)
north_arrow.setPicturePath(QgsApplication.defaultThemePath() + "/north_arrows/layout_north_arrow_01.svg")
north_arrow.setFixedSize(QgsLayoutSize(20, 20))
north_arrow.attemptMove(QgsLayoutPoint(170, 150))
layout.addLayoutItem(north_arrow)
```

- Uses a built-in QGIS SVG image of a north arrow.
- If it doesn't appear, you can replace the path with your own SVG file.

I. Show CRS Information (e.g., EPSG:4326)

```
crs_label = QgsLayoutItemLabel(layout)
crs_text = "CRS: " + layer.crs().authid()
crs_label.setText(crs_text)
crs_label.setFont(QFont("Arial", 9))
crs_label.adjustSizeToText()
crs_label.attemptMove(QgsLayoutPoint(20, 165))
layout.addLayoutItem(crs_label)
```

- Shows the Coordinate Reference System used by the map.
- Example: CRS: EPSG: 4326.

J. Export the Map as an Image

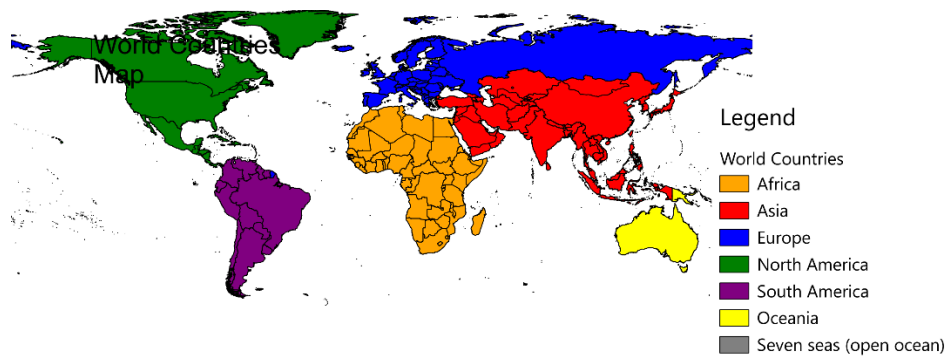
```
1 exporter = QgsLayoutExporter(layout)
2 output_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\world_map_complete.png"
3 result = exporter.exportToImage(output_path, QgsLayoutExporter.ImageExportSettings())
4
```

- Exports the entire layout as a .png file to the specified path.

K. Print Export Result

```
1 ✓ if result == QgsLayoutExporter.Success:
2     print(" Map with all elements exported successfully to:", output_path)
3 ✓ else:
4     print(" Export failed.")
5
```

- Confirmation message printed in the Python Console.



0

CRS:
EPSG:4326

Part 3: Spatial Data Analysis with Python

Spatial Data Analysis with Python Spatial data analysis is one of the most powerful components of Geographic Information Systems (GIS). In this section, we explore how Python, especially when combined with QGIS or other geospatial libraries, can be leveraged to perform essential spatial analysis tasks. These analyses help uncover patterns, relationships, and meaningful insights from geospatial datasets.

We focus on three core spatial data analysis techniques:

3.1. Terrain and Slope Analysis

Terrain analysis using Digital Elevation Models (DEMs) is crucial for understanding surface properties such as slope, aspect, and elevation. This kind of analysis can support hydrological modeling, land use planning, and construction design. Using Python libraries like GDAL, rasterio, or directly through PyQGIS, one can automate slope map generation, watershed delineation, and hillshade modeling.

Let's walk through terrain and slope analysis using Python in QGIS with a Digital Elevation Model (DEM). You'll be using:

■ Folder: D:\QGIS Tutorials\DEM

■ Data file: ASDEM_dem.tif

We'll perform:

- Load the DEM
- Generate a hillshade
- Calculate slope
- Style the layers
- Add the outputs to the QGIS project

Terrain and Slope Analysis (Full Python Code)

```

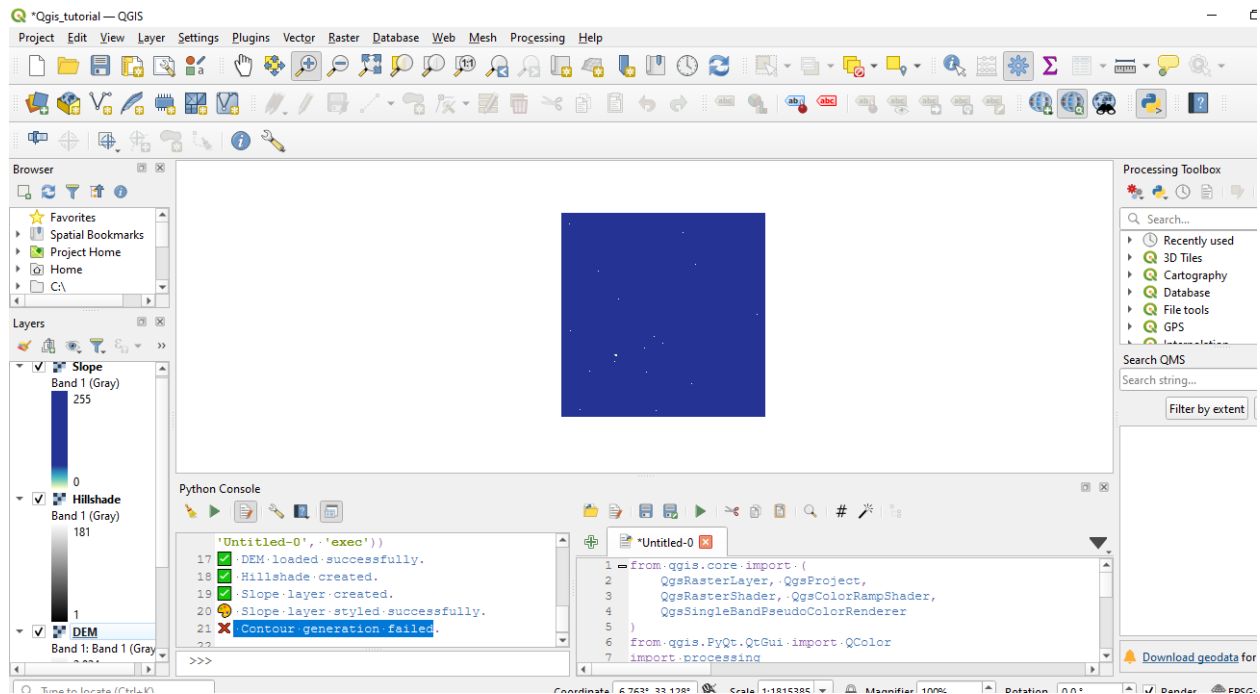
1  from qgis.core import (
2      QgsRasterLayer, QgsProject,
3      QgsRasterShader, QgsColorRampShader,
4      QgsSingleBandPseudoColorRenderer
5  )
6  from qgis.PyQt.QtGui import QColor
7  import processing
8
9  # 1. Load DEM
10 dem_path = r"D:\QGIS Tutorials\DEM\ASDEM_dem.tif"
11 dem_layer = QgsRasterLayer(dem_path, "DEM")
12 v if not dem_layer.isValid():
13     print("✗ Failed to load DEM.")
14 v else:
15     QgsProject.instance().addMapLayer(dem_layer)
16     print("✓ DEM loaded successfully.")
17
18 # 2. Generate Hillshade Layer
19 hillshade_path = r"D:\QGIS Tutorials\DEM\ASDEM_hillshade.tif"
20 v processing.run("gdal:hillshade", {
21     'INPUT': dem_path,
22     'BAND': 1,
23     'Z_FACTOR': 1,
24     'SCALE': 1,
25     'AZIMUTH': 315,
26     'ALTITUDE': 45,
27     'COMPUTE_EDGES': True,
28     'ZEVENBERGEN': False,
29     'MULTIDIRECTIONAL': False,
30     'OUTPUT': hillshade_path
31 })
32
33 hillshade_layer = QgsRasterLayer(hillshade_path, "Hillshade")
34 v if hillshade_layer.isValid():
35     QgsProject.instance().addMapLayer(hillshade_layer)
36     print("✓ Hillshade created.")
37 v else:
38     print("✗ Hillshade creation failed.")
39

```

```

40 # 3. Generate Slope Layer
41 slope_path = r"D:\QGIS Tutorials\DEM\ASDEM_slope.tif"
42 v processing.run("gdal:slope", {
43     'INPUT': dem_path,
44     'BAND': 1,
45     'SCALE': 1,
46     'AS_PERCENT': False,
47     'COMPUTE_EDGES': True,
48     'ZEVENBERGEN': False,
49     'OUTPUT': slope_path
50 })
51
52 slope_layer = QgsRasterLayer(slope_path, "Slope")
53 v if slope_layer.isValid():
54     QgsProject.instance().addMapLayer(slope_layer)
55     print("✅ Slope layer created.")
56 v else:
57     print("❌ Slope creation failed.")
58
59 # 4. Apply Slope Color Ramp Styling
60 shader = QgsRasterShader()
61 color_ramp = QgsColorRampShader()
62 color_ramp.setColorRampType(QgsColorRampShader.Interpolated)
63 v color_ramp.setColorRampItemList([
64     QgsColorRampShader.ColorRampItem(0, QColor("#ffffcc"), "Low"),
65     QgsColorRampShader.ColorRampItem(15, QColor("#a1dab4"), "Moderate"),
66     QgsColorRampShader.ColorRampItem(30, QColor("#41b6c4"), "High"),
67     QgsColorRampShader.ColorRampItem(45, QColor("#2c7fb8"), "Very High"),
68     QgsColorRampShader.ColorRampItem(60, QColor("#253494"), "Extreme")
69 ])
70 shader.setRasterShaderFunction(color_ramp)
71
72 slope_renderer = QgsSingleBandPseudoColorRenderer(slope_layer.dataProvider(), 1, shader)
73 slope_layer.setRenderer(slope_renderer)
74 slope_layer.triggerRepaint()
75
76 print("👉 Slope layer styled successfully.")
77

```



3.2. Clipping, Buffering, and Intersecting Features

These are common vector operations used in spatial analysis. Buffering creates a zone around features (e.g., roads, rivers), while clipping and intersecting help extract or combine spatial features based on their geographic boundaries. These operations allow users to define areas of interest and perform targeted analyses, such as identifying all schools within 1 km of a major road.

Clipping

In Geographic Information Systems (GIS), **clipping** is a fundamental spatial operation used to extract features of interest from a larger dataset using a defined boundary. Think of it like using a cookie cutter on a sheet of dough the cutter represents your area of interest (AOI), and the result is a neatly shaped extract of data.

In this example, we demonstrate how to clip **Ethiopia's river network** using the **Benchmaji zone boundary**, effectively isolating all rivers that flow within that administrative region.

Its objectives is To extract only the rivers located inside the Benchmaji zone, using the national river shapefile as the source and the zone boundary as the mask.

A. *Clipping helps us:*

- Focus your analysis on a specific area
- Reduce file size and complexity
- Improve visualization for localized studies
- Prepare data for zone-based analysis and map production

B. *Input Datasets*

✓ **River shapefile:**

hotosm_eth_waterways_lines_shp.shp

(Located at: D:\QGIS Tutorials\ethio rivers)

✓ **Benchmaji zone boundary shapefile:**

benchmaji.shp

(Located at: D:\QGIS Tutorials\zone)

C. Using Python (GeoPandas)

```
1  import geopandas as gpd
2
3  # File paths
4  river_fp = r"D:\QGIS Tutorials\ethio rivers\hotosm_eth_waterways_lines_shp.shp"
5  zone_fp = r"D:\QGIS Tutorials\zone\benchmaji.shp"
6  output_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"
7
8  # Load the data
9  rivers = gpd.read_file(river_fp)
10 zone = gpd.read_file(zone_fp)
11
12 # Ensure same CRS
13 rivers = rivers.to_crs(zone.crs)
14
15 # Clip rivers using Benchmaji boundary
16 clipped_rivers = gpd.clip(rivers, zone)
17
18 # Save result
19 clipped_rivers.to_file(output_fp)
20
21 print("✅ Rivers clipped successfully to Benchmaji zone!")
22
```

```

# File path to the clipped river shapefile
clipped_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"

# Load the layer into QGIS map canvas
layer = iface.addVectorLayer(clipped_fp, "Benchmaji Rivers", "ogr")

# Check if it was loaded successfully
if not layer:
    print("❌ Failed to load the layer.")
else:
    print("✅ Clipped river layer loaded into the map canvas.")

```

Code explanation

I. Importing geopandas

```
import geopandas as gpd
```

`geopandas` is a Python library that extends `pandas` for working with geographic data (like shapefiles). It makes it easy to load, analyze, and manipulate spatial data.

II. Defining the file path

```

# Define the file path to the river shapefile
river_fp = r"D:\QGIS Tutorials\ethio rivers\hotosm_eth_waterways_lines.shp"

# Define the file path to the Benchmaji zone boundary shapefile
zone_fp = r"D:\QGIS Tutorials\zone\benchmaji.shp"

# Define the file path where the output (clipped river data) will be saved
output_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"

```

We use raw string format (`r"..."`) to avoid errors with backslashes in file paths. Each variable holds the full location of the shapefiles on your computer:

- `river_fp`: the complete river network of Ethiopia
- `zone_fp`: the administrative boundary of Benchmaji zone
- `output_fp`: the destination file for saving the clipped result

III. Read the shapfiles

```
# Read the river shapefile using GeoPandas
rivers = gpd.read_file(river_fp)

# Read the Benchmaji boundary shapefile
zone = gpd.read_file(zone_fp)
```

`read_file ()` is used to load shapefiles into GeoDataFrames (`rivers` and `zone`). Each GeoDataFrame behaves like an Excel table, but with geometry (map shapes) attached to each row.

IV. Check the Coordinate System

```
# Perform the clipping operation: keep only rivers that are inside the Benchmaji zone
clipped_rivers = gpd.clip(rivers, zone)
```

Before spatial operations like clipping, both layers must be in the same CRS (i.e., same map projection). This line ensures that the `rivers` layer is reprojected to match the CRS of `zone`.

V. Keep only rivers that are inside the Benchmaji zone

This line clips the river lines, keeping only the portions inside the `Benchmaji zone`. The result is stored in a new GeoDataFrame called `clipped_rivers`.

```
# Perform the clipping operation: keep only rivers that are inside the Benchmaji zone
clipped_rivers = gpd.clip(rivers, zone)
```

VI. Save the clipped result and Inform the user that the process is complete

The result is saved as a new shapefile (`benchmaji_rivers.shp`) at the path you defined earlier. This file can now be loaded into QGIS.

```
# Save the clipped result to a new shapefile
clipped_rivers.to_file(output_fp)
```

This line simply prints a success message so the user knows the script finished without errors.

```
# Inform the user that the process is complete
print("✅ Rivers clipped successfully to Benchmaji zone!")
```

VII. Define the path to the clipped river shapefile

```
# Define the path to the clipped river shapefile
clipped_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"
```

This line sets the full file path to the river shapefile you clipped earlier. We use the raw string `r"..."` format to avoid backslash errors on Windows.

VIII. Load the shapefile into the QGIS map canvas and Check if the layer loaded successfully

```
# Load the shapefile into the QGIS map canvas
layer = iface.addVectorLayer(clipped_fp, "Benchmaji Rivers", "ogr")
```

`iface.addVectorLayer(...)` is the QGIS function to load a vector file (like shapefile) directly into the map.

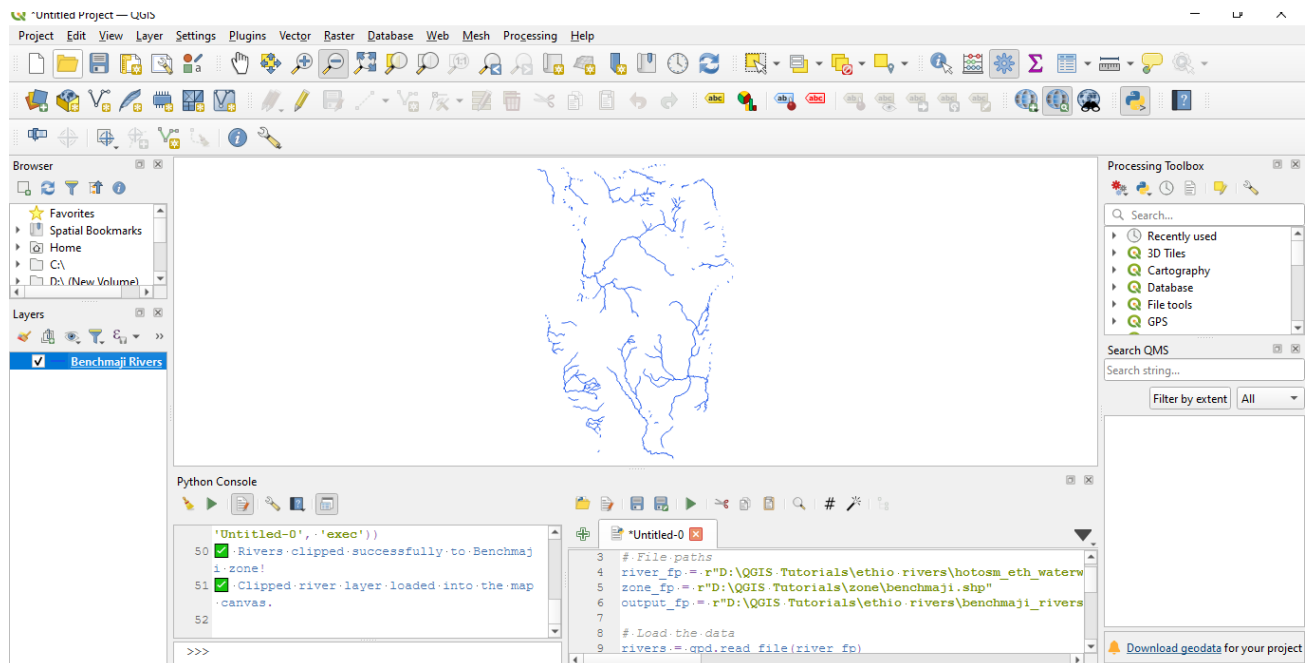
`"Benchmaji Rivers"` is the name you want to appear in the Layers panel.

`"ogr"` tells QGIS to use the OGR driver, which supports shapefiles, GeoJSON, and other formats.

After this line runs, the layer will appear in your QGIS map.

```
# Check if the Layer loaded successfully
if not layer:
    print("❌ Failed to load the layer.")
else:
    print("✅ Clipped river layer loaded into the map canvas.")
```

- ✓ This block checks if the layer was loaded successfully:
- ✓ If it fails, it prints an error message.
- ✓ If successful, it confirms with a green checkmark message.



Buffering

Buffering is a spatial operation that creates a zone around a geographic feature. It's used to identify areas of influence for example:

- ✓ Creating a 100-meter zone along rivers to study riparian vegetation
- ✓ Identifying hazard zones around fault lines
- ✓ Planning development setbacks from roads or water bodies

Input File

- ✓ The clipped river data: *benchmaji_rivers.shp*
- ✓ Located at: *D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp*
- ✓ Generate a **buffer of 500 meters** on both sides of the river.
- ✓ Save the result as a **new shapefile**.
- ✓ Load it into **QGIS** for visualization.

```

1  # Step 1: Import GeoPandas for spatial operations
2  import geopandas as gpd
3
4  # Step 2: Define the file paths
5  input_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"
6  output_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_river_buffer.shp"
7
8  # Step 3: Load the river shapefile
9  rivers = gpd.read_file(input_fp)
10
11 # Step 4: Re-project to a metric CRS (if not already)
12 # We use EPSG:32637 (UTM Zone 37N) - suitable for southwest Ethiopia
13 rivers = rivers.to_crs(epsg=32637)
14
15 # Step 5: Create a 500-meter buffer around each river line
16 buffered = rivers.buffer(500)
17
18 # Step 6: Convert the buffer result into a GeoDataFrame
19 buffered_gdf = gpd.GeoDataFrame(geometry=buffered, crs=rivers.crs)
20
21 # Step 7: Save the buffered zones as a new shapefile
22 buffered_gdf.to_file(output_fp)
23
24 # Step 8: Notify the user
25 print("✅ Buffer zones created and saved successfully!")
26
27
28 buffer_layer = iface.addVectorLayer(output_fp, "Benchmaji River Buffer", "ogr")
29 if not buffer_layer:
30     print("❌ Buffer layer failed to load.")
31 else:
32     print("✅ Buffer layer loaded into QGIS.")

```

I. Import Required Library

```
import geopandas as gpd
```

`geopandas` is the Python library we use for reading shapefiles and running spatial operations like buffering.

II. File Paths

```
input_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_rivers.shp"
output_fp = r"D:\QGIS Tutorials\ethio rivers\benchmaji_river_buffer.shp"
```

These lines define the input (clipped river shapefile) and output (buffer result) file locations.

III. Load the River Data

```
rivers = gpd.read_file(input_fp)
```

Loads the clipped **Benchmaji river** data into a GeoDataFrame.

IV. Reproject to a Metric CRS

```
rivers = rivers.to_crs(epsg=32637)
```

Buffers must be in meters, not degrees. So, we convert the projection to UTM Zone 37N, which covers southwest Ethiopia.

If you buffer in degrees (EPSG: 4326), the buffer size (like 500) would be in degrees — not useful.

V. Perform Buffering

```
buffered = rivers.buffer(500)
```

- ✓ This creates a 500-meter buffer around each river line.
- ✓ The result is a GeoSeries of polygon geometries.

VI. Convert to GeoDataFrame

```
buffered_gdf = gpd.GeoDataFrame(geometry=buffered, crs=rivers.crs)
```

Converts the buffer results into a full GeoDataFrame, so we can save/export it.

VII. Save the Result and Print Completion Message

```
buffered_gdf.to_file(output_fp)
```

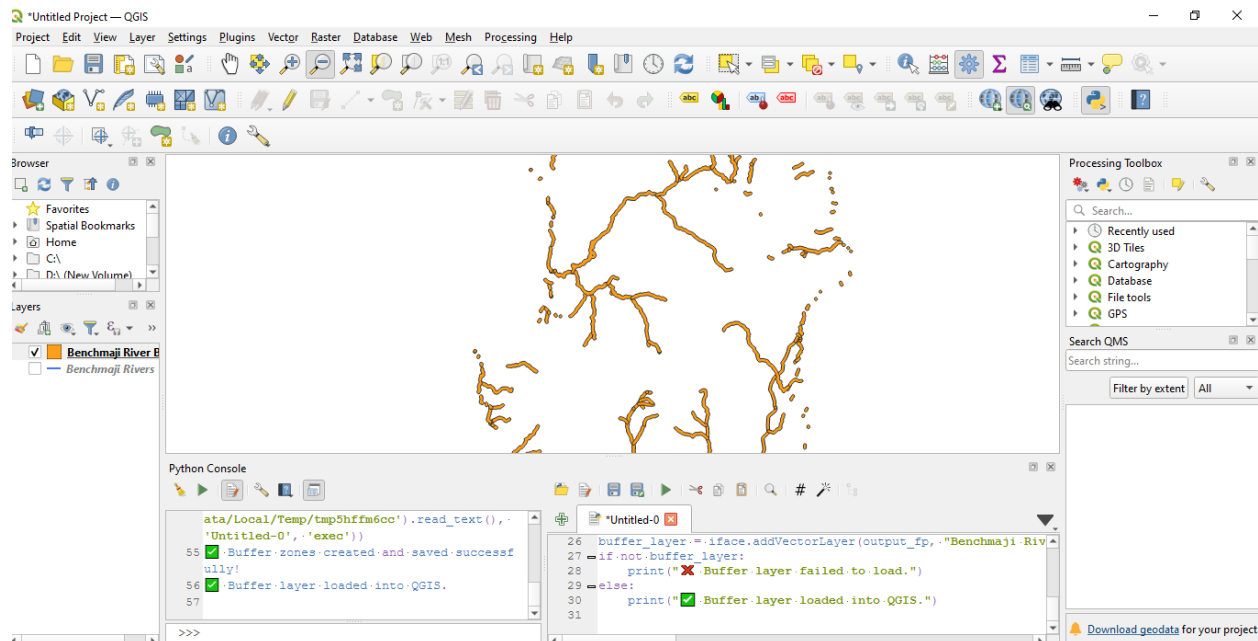
Saves the buffered data to a new shapefile for use in QGIS or other GIS software.

```
print("✅ Buffer zones created and saved successfully!")
```

Just a friendly message confirming that the script worked!

VIII. Optional: Load into QGIS Map Canvas

```
buffer_layer = iface.addVectorLayer(output_fp, "Benchmaji River Buffer", "ogr")
if not buffer_layer:
    print("❌ Buffer layer failed to load.")
else:
    print("✅ Buffer layer loaded into QGIS.")
```



Intersecting Features

Intersecting Features in GIS Intersection is a spatial analysis operation used to identify and extract the geometric overlap between two layers. The result is a new layer containing only the features (or parts of features) that are common to both input layers.

◆ Purpose of Intersecting Features To find shared areas between two datasets (e.g., roads within urban zones, protected areas intersecting agricultural land).

To perform advanced spatial queries that require both spatial and attribute criteria.

To create new datasets for further analysis, such as suitability modeling or impact assessment.

◆ Input Requirements Both input layers must be vector datasets.

Common geometry types (e.g., polygon & polygon, line & polygon) depending on the analysis context.

Coordinate Reference Systems (CRS) should ideally match to avoid alignment issues.

◆ Result A new vector layer (e.g., shapefile) that contains only the overlapping geometries and combined attribute information from both layers.

◆ Common Applications Identifying land parcels that fall within flood zones.

Finding road segments within city boundaries.

Mapping deforestation within protected areas.

Analyzing population exposure to hazard zones.

3.3. Calculating Statistics and Attribute-Based Filtering

This involves using attribute tables and spatial queries to extract meaningful statistics. Python can be used to summarize data (e.g., mean population density, land cover distribution) or to filter features based on attribute conditions. Combined with pandas, geopandas, or QGIS field calculator through PyQGIS, this method can automate reports and visualizations of spatial trends.

Each of these analytical techniques can be performed interactively within QGIS or entirely through Python scripts, enabling automation and reproducibility. These tools are foundational for more advanced geospatial modeling and decision-making processes.

Example To analyze population data on a world map to:

Calculate summary statistics such as total population, mean population, or population density.

Filter countries based on attribute conditions, e.g., countries with population greater than 100 million.

Required Data

A vector shapefile of world countries (e.g., world_population.shp).

The shapefile must include attributes like:

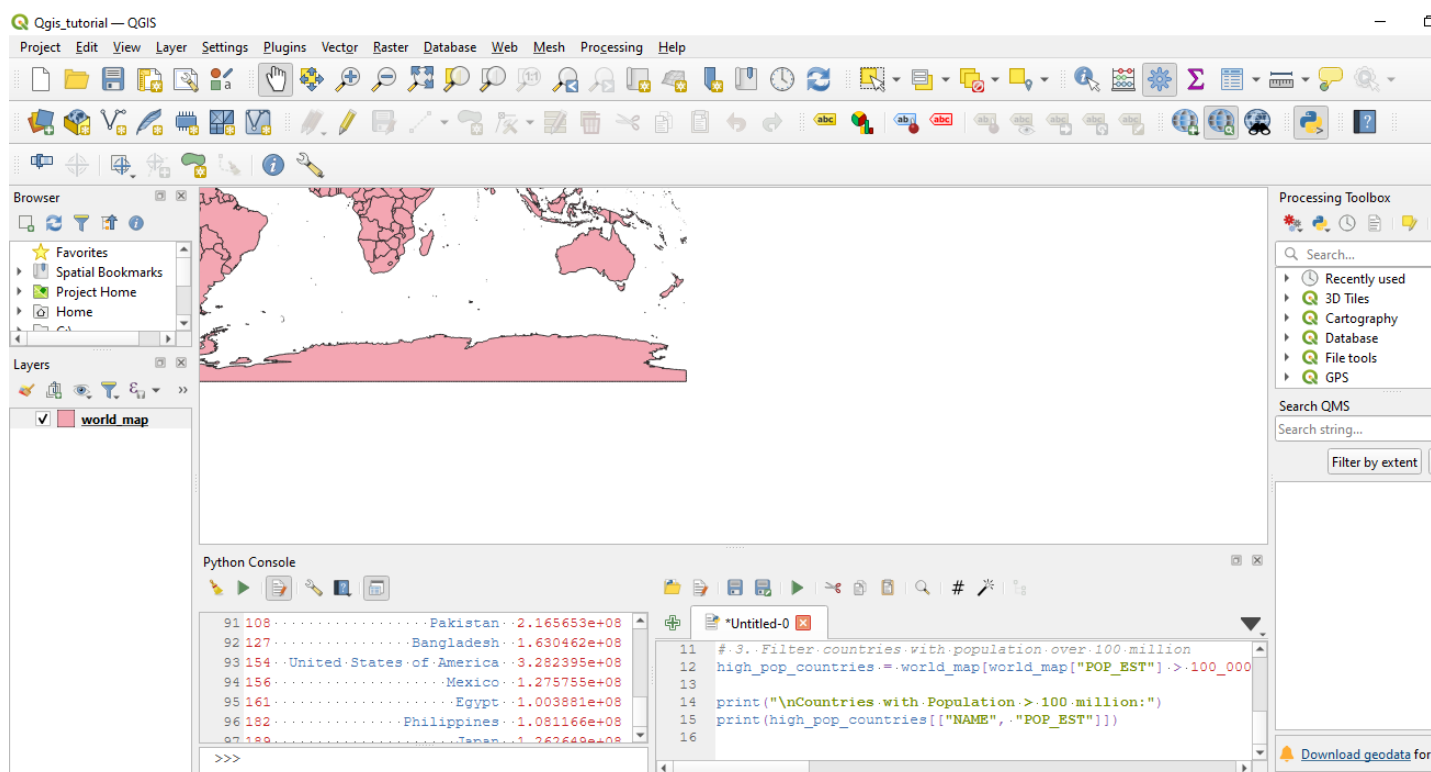
POP_EST (Population estimate)

NAME (Country name)

AREA or geometry (for density calculation)

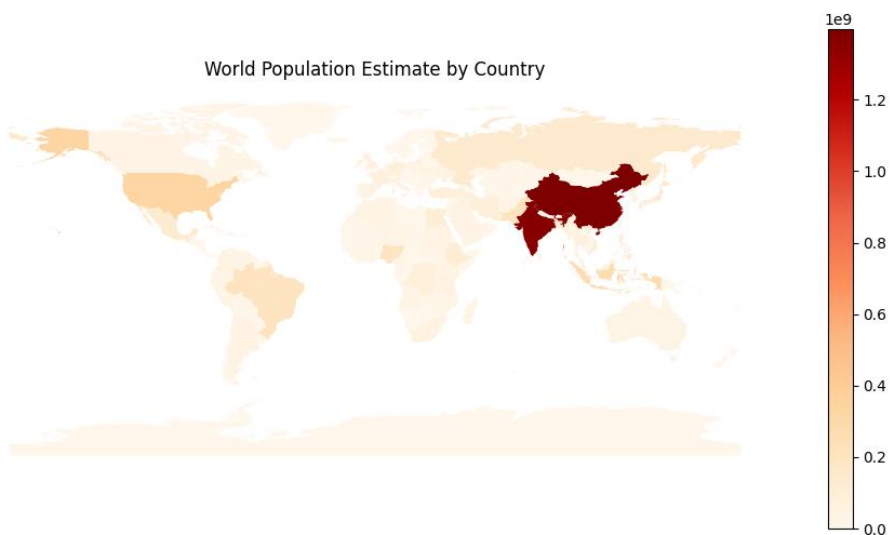
Here's how you can perform attribute-based filtering and calculate statistics using that column in Python (with GeoPandas):

```
1 import geopandas as gpd
2
3 # 1. Load the shapefile
4 shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
5 world_map = gpd.read_file(shapefile_path)
6
7 # 2. Show basic population statistics
8 print("Population Estimate Statistics:")
9 print(world_map["POP_EST"].describe())
10
11 # 3. Filter countries with population over 100 million
12 high_pop_countries = world_map[world_map["POP_EST"] > 100_000_000]
13
14 print("\nCountries with Population > 100 million:")
15 print(high_pop_countries[["NAME", "POP_EST"]])
16
```



World population estimation by country+++++++

```
1  import geopandas as gpd
2
3  # Your already-loaded GeoDataFrame
4  # world_map = gpd.read_file("path_to_your_file") # Already loaded, so not needed again
5
6  # 1. View basic statistics of the POP_EST column
7  print("Basic Statistics:")
8  print(world_map["POP_EST"].describe())
9
10 # 2. Filter countries with population greater than 100 million
11 pop_over_100m = world_map[world_map["POP_EST"] > 100_000_000]
12
13 print("\nCountries with population > 100 million:")
14 print(pop_over_100m[["NAME", "POP_EST"]])
15
16 # 3. Total world population estimate (based on POP_EST)
17 total_population = world_map["POP_EST"].sum()
18 print(f"\nEstimated Total World Population: {total_population:,}")
19
20 # 4. Plot the population distribution
21 import matplotlib.pyplot as plt
22
23 world_map.plot(column="POP_EST", legend=True, cmap="OrRd", figsize=(12, 6))
24 plt.title("World Population Estimate by Country")
25 plt.axis("off")
26 plt.show()
27
```



3.4. Advanced Data Handling

As geospatial projects expand in scale and complexity, so does the need for efficient and intelligent data management. In this part of the series, we focus on advanced data handling techniques using Python primarily through PyQGIS and GeoPandas. While QGIS offers a rich graphical interface, scripting your data operations makes it easier to automate repetitive tasks, ensure consistency, and handle large or messy datasets with precision.

One major focus of this part is working with GeoPandas, a Python library that extends the capabilities of pandas for geospatial data. GeoPandas provides a user-friendly way to load, manipulate, and export spatial files like Shapefiles and GeoJSON without needing to open QGIS. We'll compare and combine its use with PyQGIS, especially when we need to move data between the QGIS environment and Python-based analysis workflows.

We also explore how to connect Python with external data sources, including CSV, Excel, and relational databases. This is especially valuable when spatial data must be enriched with demographic, environmental, or financial data stored in tabular formats. You'll learn how to read these external files, link them to vector layers using common keys, and write custom scripts for live data ingestion or batch processing.

Finally, the section covers how to read and write spatial files using Python. You'll learn how to create new shapefiles or GeoPackages from scratch, modify geometries or attributes, and save updates—all programmatically. These skills are vital for building dynamic GIS applications, automating field data collection pipelines, or preprocessing data for web-based maps.

By the end of this part, you'll have a strong command of advanced data workflows that integrate GIS with the broader Python data science ecosystem. This opens the door to scalable, flexible, and professional-grade solutions for academic research, government planning, or private sector applications.

3.5. Connecting Python with External Data (CSV, Excel, Databases)

In many GIS workflows, spatial data must be enriched with non-spatial attributes stored in formats like CSV, Excel spreadsheets, or relational databases. For instance, you may have a shapefile of administrative boundaries and a separate CSV containing population or health data. This section focuses on how to read, clean, and join such external data sources using Python—and then link them to vector layers in QGIS using PyQGIS or GeoPandas.

We'll cover how to read CSV and Excel files using pandas, how to merge them with shapefiles using common attributes (like worldcities names or codes), and how to establish database connections for more advanced, scalable projects. This is especially useful for automating data pipelines or building dashboards that rely on frequently updated tabular data.

Reading CSV and Excel in Python

First, make sure your worldcities.csv file contains **latitude** and **longitude** columns. Usually they are named something like lat, lng, latitude, longitude, etc

```
1  import pandas as pd
2  import geopandas as gpd
3  from shapely.geometry import Point
4
5  # Step 1: Load the CSV using pandas
6  csv_file = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\worldcities.csv"
7  df = pd.read_csv(csv_file)
8
9  # Step 2: Convert to GeoDataFrame using latitude and longitude
10 # Replace 'lat' and 'lng' with your actual column names if different
11 geometry = [Point(xy) for xy in zip(df['lng'], df['lat'])]
12 gdf = gpd.GeoDataFrame(df, geometry=geometry)
13
14 # Step 3: Set coordinate reference system (usually WGS84)
15 gdf.set_crs(epsg=4326, inplace=True)
16
17 # Optional: Preview
18 print(gdf.head())
19
```

To Save the GeoDataFrame as a Shapefile

```
output_file = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\worldcities.shp"
gdf.to_file(output_file)
```

To Visualize

```
import matplotlib.pyplot as plt

gdf.plot(markersize=5, color='red')
plt.title("World Cities")
plt.show()
```

Let's Breakdown the code what is the functions of each code.

Step 1. Import Required Libraries

```
import pandas as pd          # For reading and handling CSV data
import geopandas as gpd      # For spatial data handling
from shapely.geometry import Point # For creating geometry (points)
```

- pandas is used to read CSV files and manipulate tabular data.
- geopandas builds on pandas and adds spatial/geographic functionality.
- Point from shapely.geometry is used to convert coordinate pairs (longitude, latitude) into actual spatial points.

Step 2. Read the CSV File

```
csv_file = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\worldcities.csv"
df = pd.read_csv(csv_file)
```

- `pd.read_csv()` reads the CSV file into a DataFrame.
- `r` before the string ensures that backslashes (`\`) in Windows file paths are treated correctly.

Step 3: Check Column Names (Optional but helpful)

```
print(df.columns)
```

- This shows all the column names. Look for columns like `lat`, `lng`, `latitude`, or `longitude` to identify location data.

Step 4: Convert the DataFrame into a GeoDataFrame

```
geometry = [Point(xy) for xy in zip(df['lng'], df['lat'])] # Adjust column names if needed
gdf = gpd.GeoDataFrame(df, geometry=geometry)
```

- `zip(df['lng'], df['lat'])` pairs each row's longitude and latitude.
- `Point(xy)` converts each pair into a geometry object.
- `GeoDataFrame` combines tabular data with the geometry to create a spatial object.

Step 5: Set the Coordinate Reference System (CRS)

```
gdf.set_crs(epsg=4326, inplace=True)
```

- EPSG:4326 refers to WGS 84, the most common geographic coordinate system (used in GPS).
- It defines what the coordinate numbers (lat/lon) actually mean on the Earth's surface.

Step 6: Preview the Spatial Data (Optional)

```
print(gdf.head())
```

- Displays the first 5 rows of your spatial data to confirm it loaded correctly.

Step 7: Save the GeoDataFrame as a Shapefile

```
output_file = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\worldcities.shp"
gdf.to_file(output_file)
```

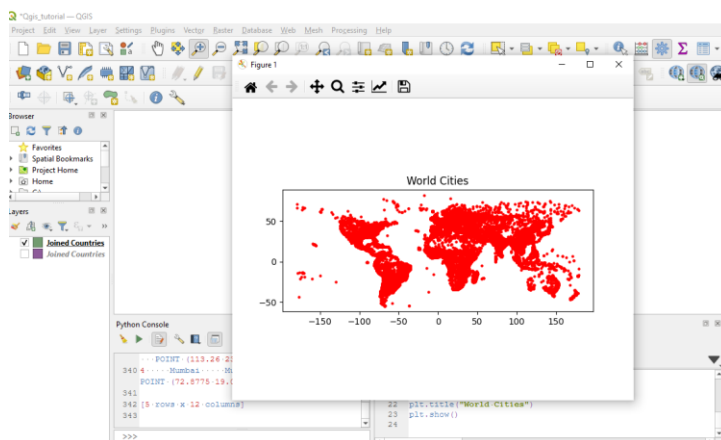
- Saves the spatial data in shapefile format so you can use it in QGIS or any GIS software.

Step 8: Plot the Points on a Map (Optional Visualization)

```
import matplotlib.pyplot as plt

gdf.plot(markersize=5, color='red')
plt.title("World Cities")
plt.show()
```

- plot() draws the point locations of cities.
- markersize controls the dot size.
- color='red' sets the marker color.
- plt.show() displays the plot window.



Joining CSV with GeoPandas Layer

This tutorial demonstrates how to join a tabular dataset (CSV file) with a shapefile (vector data) using GeoPandas in Python, and how to visualize the result. This is useful for creating thematic maps, such as GDP or population maps.

```
1  import geopandas as gpd
2  import pandas as pd
3  import matplotlib.pyplot as plt
4  import os
5  import tempfile
6
7  # Paths
8  shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
9  csv_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\sample_country_data.csv"
10
11 # Load shapefile
12 world_map = gpd.read_file(shapefile_path)
13
14 # Load CSV
15 csv_data = pd.read_csv(csv_path)
16
17 # Check available columns
18 print("Shapefile columns:", world_map.columns)
19 print("CSV columns:", csv_data.columns)
20
21 # Ensure common column name for joining
22 # Assume shapefile uses 'NAME' and CSV uses 'Country'
23 csv_data.rename(columns={'Country': 'NAME'}, inplace=True)
```

```

24
25 # Merge data
26 joined = world_map.merge(csv_data, on='NAME')
27
28 # Show a preview
29 print(joined.head())
30
31 # Plot using one column from the CSV (e.g., 'GDP_Billions')
32 joined.plot(
33     column='GDP_Billions', # Replace with any column you want to visualize
34     legend=True,
35     cmap='OrRd',
36     edgecolor='black'
37 )
38
39 plt.title("World Map Colored by GDP (Billions)")
40 plt.axis('off')
41 plt.show()
42
43 # ----- OPTIONAL: Display result in QGIS if run from QGIS -----
44 v try:
45     from qgis.utils import iface
46     from qgis.core import QgsVectorLayer, QgsProject
47
48     # Save to temporary shapefile
49     temp_file = os.path.join(tempfile.gettempdir(), "joined_output.shp")
50     joined.to_file(temp_file)
51
52     # Add to QGIS
53     layer = iface.addVectorLayer(temp_file, "Joined Countries", "ogr")
54 v if not layer:
55     print("Layer failed to load!")
56 v else:
57     print("Layer added to QGIS successfully.")
58
59 v except ImportError:
60     print("Not running inside QGIS. Skipping QGIS display step.")
61

```

Step 1. Import Required Libraries

```

import geopandas as gpd
import pandas as pd
import matplotlib.pyplot as plt
import os
import tempfile

```

- geopandas: Used for handling spatial vector data.
- pandas: For loading and processing the CSV file.
- matplotlib.pyplot: For plotting the final map.
- os, tempfile: To manage file paths and create a temporary shapefile if using QGIS.

🗺️ Step 2. Define File Paths

```
shapefile_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\ne_10m_admin_0_countries.shp"
csv_path = r"D:\QGIS Tutorials\ne_10m_admin_0_countries\sample_country_data.csv"
```

- This step specifies the location of your input shapefile and CSV file.

📂 Step 3. Load the Spatial and Tabular Data

```
world_map = gpd.read_file(shapefile_path)
csv_data = pd.read_csv(csv_path)
```

- read_file(): Loads the shapefile as a GeoDataFrame.
- read_csv(): Loads the CSV file as a DataFrame.

🔍 Step 4. Check the Column Names

```
print("Shapefile columns:", world_map.columns)
print("CSV columns:", csv_data.columns)
```

- Before merging, it's important to confirm the key column names you'll join on.
- For example, shapefile might have "NAME" while CSV might use "Country".

✂️ Step 5. Rename Column in CSV to Match Shapefile

```
csv_data.rename(columns={'Country': 'NAME'}, inplace=True)
```

- This makes sure both files have a common join key ("NAME").

🔗 Step 6. Join the CSV and Shapefile

```
joined = world_map.merge(csv_data, on='NAME')
```

- Performs an attribute join on the NAME column.
- The result is a new GeoDataFrame with combined geometry and CSV attributes.

Step 7. Preview the Joined Data

```
print(joined.head())
```

- Displays the first 5 rows to confirm the join worked.

Step 8. Plot the Result

```
joined.plot(
    column='GDP_Billions',
    legend=True,
    cmap='OrRd',
    edgecolor='black'
)

plt.title("World Map Colored by GDP (Billions)")
plt.axis('off')
plt.show()
```

- column: Specifies the attribute to visualize (e.g., GDP).
- legend=True: Adds a legend to the map.
- cmap: Chooses a color scheme (OrRd = Orange-Red).
- edgecolor='black': Adds black borders to countries.
- plt.show(): Displays the map.

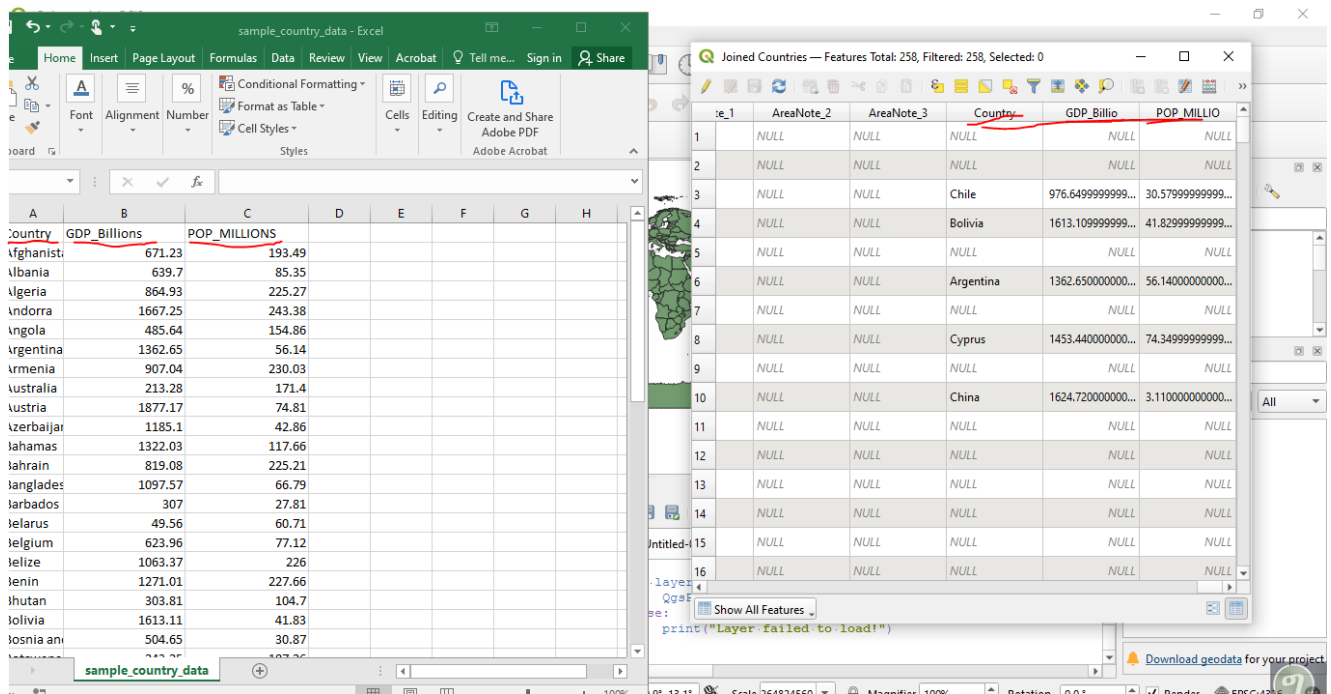
Step 9. (Optional) Add Result to QGIS (If Using PyQGIS)

```
try:
    from qgis.utils import iface
    from qgis.core import QgsVectorLayer, QgsProject

    temp_file = os.path.join(tempfile.gettempdir(), "joined_output.shp")
    joined.to_file(temp_file)

    layer = iface.addVectorLayer(temp_file, "Joined Countries", "ogr")
    if not layer:
        print("Layer failed to load!")
    else:
        print("Layer added to QGIS successfully.")
except ImportError:
    print("Not running inside QGIS. Skipping QGIS display step.")
```

- This block saves the merged result to a shapefile and loads it into QGIS.
- Only works if you're running this script from the **QGIS Python Console**



Part 4: Real-World Projects

This part of the series is where theory meets practice. After exploring the foundations of QGIS and Python, it's time to apply everything you've learned in real-world GIS projects. This section is designed to simulate the kind of tasks GIS professionals face in the field—integrating spatial analysis, data handling, visualization, and automation into complete workflows. These projects will help reinforce your knowledge while giving you a portfolio of practical examples.

Each project will focus on a specific problem-solving scenario such as environmental monitoring, infrastructure planning, disaster risk assessment, or demographic analysis. We'll walk through each step: from loading and preparing spatial and non-spatial data, conducting relevant analysis, to visualizing results in professional maps or dashboards. These hands-on examples serve as templates you can adapt and reuse in your own work.

We'll use both GeoPandas and PyQGIS, depending on the project's nature—sometimes leveraging the speed and flexibility of Python scripts outside of QGIS, and other times using QGIS's full desktop power via PyQGIS scripting. Some projects may include time-saving features such as automated map generation, batch processing, or linking to external data sources like APIs or spreadsheets.

By the end of this section, you won't just know how QGIS and Python work—you'll be ready to build robust GIS tools and workflows that solve real challenges, support decision-making, and provide value in academic, governmental, or business contexts.

Land Cover Change Detection

✓ Data Types:

Raster datasets (e.g., satellite images: Landsat, Sentinel-2, etc.)

Optional vector boundaries (e.g., study area shapefiles)

🔧 Methods:

Image preprocessing (clipping, reprojecting)

Band combinations or NDVI calculation

Raster classification (e.g., unsupervised or supervised classification using KMeans, Random Forest)

Change detection (using raster differencing, image comparison, or pixel-based change analysis)

Tools/Libraries:

`rasterio`, `numpy`, `matplotlib`, `sklearn`, `geopandas`, `PyQGIS`, and optionally `scikit-image`

Site Suitability Analysis

✓ Data Types:

Raster layers (e.g., slope, land use, distance to roads or rivers)

Vector data (e.g., administrative boundaries, urban centers)

🔧 Methods:

Raster reclassification and normalization

Weight assignment using Multi-Criteria Decision Analysis (MCDA)

Weighted overlay analysis using raster algebra

Masking and extraction within regions of interest

Tools/Libraries:

`rasterio`, `numpy`, `geopandas`, `matplotlib`, `scipy`, `PyQGIS`

Population Density Mapping Using Raster Data

Data Types:

Gridded population datasets (e.g., WorldPop, GPWv4, or census raster layers)

Vector boundaries (e.g., admin districts)

Optional infrastructure layers (e.g., roads, health centers)

🔧 Methods:

Raster aggregation or disaggregation

Zonal statistics to summarize population by region

Reprojection and resolution matching

Visualization with choropleth maps

Tools/Libraries:

```
rasterstats, geopandas, rasterio, xarray, matplotlib, PyQGIS
```

Land Cover Change Detection Using Landsat 8 Data (2015–2024)

This project aims to identify and visualize land cover changes over a 9-year period using multi-temporal Landsat 8 imagery. Specifically, we compare satellite images from 2015 and 2024 to detect areas that have experienced vegetation loss or other significant land use changes.

NDVI (Normalized Difference Vegetation Index):

It is a vegetation index that measures the greenness or health of vegetation using Red (Band 4) and Near Infrared (NIR, Band 5) wavelengths from satellite imagery.

$$NDVI = \frac{NIR - Red}{NIR + Red}$$

NDVI values range from -1 to +1:

- Water: < 0
- Bare Soil: 0.1–0.2
- Vegetation: 0.2–0.8+

NDVI-Based Change Detection:

This method involves calculating NDVI for two different dates and subtracting one from the other to detect change in vegetation health or coverage.

$$\text{NDVI Change} = \text{NDVI}_{\text{Later}} - \text{NDVI}_{\text{Earlier}}$$

Positive Change → Increase in vegetation (e.g., afforestation, crop growth)

Negative Change → Loss of vegetation (e.g., deforestation, urbanization)

When to Use NDVI-Based Change Detection?

- Our main goal is to monitor vegetation gain/loss.
- We want to perform quick or continuous change analysis.
- We don't have training data for classification.
- We're studying a large area with significant vegetation cover.

Dataset Description

We use the following Landsat 8 Level 1 data:

- **2015:** LC08_L1TP_170055_20150204_20200910_02_T1
- **2024:** LC08_L1TP_170055_20240229_20240313_02_T1

Each dataset includes multiple bands.

For NDVI-based land cover change detection,

We will use:

Band 4 (Red)

Band 5 (Near Infrared - NIR)

Data Preparation

- D.** Unzip .tar files and organize bands into separate folders (2015/, 2024/).
- E.** Load the Red and NIR bands using rasterio.
- F.** Ensure both images are projected in the same Coordinate Reference System (CRS).

Step 1: Calculate NDVI for Each Image

File Path: D:\QGIS Tutorials\LULC\LC08_L1TP_170055_20150204_20200910_02_T1

The file Path: D:\QGIS Tutorials\LULC\LC08_L1TP_170055_20240229_20240313_02_T1

From the Landsat 8 to calculate NDVI Red Band = Band 4 and NIR Band = Band 5 are use.

To calculate the NDVI of 2015 and 2024 year image using QGIS and python script.

```
1  import rasterio
2  import numpy as np
3  import os
4  import matplotlib.pyplot as plt
5
6  # --- Define paths ---
7  input_dirs = [
8      r"D:\QGIS Tutorials\LULC\LC08_L1TP_170055_20150204_20200910_02_T1",
9      r"D:\QGIS Tutorials\LULC\LC08_L1TP_170055_20240229_20240313_02_T1"
10 ]
11
12 output_dir = r"D:\QGIS Tutorials\LULC\NDVI"
13 os.makedirs(output_dir, exist_ok=True)
14
15 def calculate_ndvi(red_path, nir_path, out_path, title):
16     with rasterio.open(red_path) as red_src, rasterio.open(nir_path) as nir_src:
17         red = red_src.read(1).astype('float32')
18         nir = nir_src.read(1).astype('float32')
19         profile = red_src.profile
20
21         # Avoid divide by zero
22         ndvi = (nir - red) / (nir + red + 1e-10)
23         profile.update(dtype=rasterio.float32, count=1)
24
25     with rasterio.open(out_path, 'w', **profile) as dst:
26         dst.write(ndvi, 1)
27
```

```

28         # Display
29         plt.figure(figsize=(8,6))
30         plt.imshow(ndvi, cmap='RdYlGn', vmin=-1, vmax=1)
31         plt.colorbar(label='NDVI')
32         plt.title(f"NDVI - {title}")
33         plt.axis('off')
34         plt.show()
35
36     # Process both images
37     calculate_ndvi(
38         os.path.join(input_dirs[0], 'LC08_L1TP_170055_20150204_20200910_02_T1_B4.TIF'),
39         os.path.join(input_dirs[0], 'LC08_L1TP_170055_20150204_20200910_02_T1_B5.TIF'),
40         os.path.join(output_dir, 'NDVI_2015.tif'),
41         "2015"
42     )
43
44     calculate_ndvi(
45         os.path.join(input_dirs[1], 'LC08_L1TP_170055_20240229_20240313_02_T1_B4.TIF'),
46         os.path.join(input_dirs[1], 'LC08_L1TP_170055_20240229_20240313_02_T1_B5.TIF'),
47         os.path.join(output_dir, 'NDVI_2024.tif'),
48         "2024"
49     )

```

NDVI Change Detection (2024 - 2015)

This project demonstrates a basic yet powerful approach to detect land cover change using NDVI. You we enhance this analysis by:

The python script to calculate the change detection is

```

1  import rasterio
2  import numpy as np
3  import matplotlib.pyplot as plt
4  import os
5
6  ndvi_2015_path = r"D:\QGIS Tutorials\LULC\NDVI\NDVI_2015.tif"
7  ndvi_2024_path = r"D:\QGIS Tutorials\LULC\NDVI\NDVI_2024.tif"
8  output_change_path = r"D:\QGIS Tutorials\LULC\NDVI\NDVI_Change_2024_2015.tif"
9
10 with rasterio.open(ndvi_2015_path) as src1, rasterio.open(ndvi_2024_path) as src2:
11     ndvi_2015 = src1.read(1)
12     ndvi_2024 = src2.read(1)
13     profile = src1.profile
14
15     # Calculate NDVI change
16     ndvi_change = ndvi_2024 - ndvi_2015
17
18     profile.update(dtype=rasterio.float32)
19
20     # Save change detection image
21 with rasterio.open(output_change_path, 'w', **profile) as dst:
22     dst.write(ndvi_change.astype('float32'), 1)
23
24
25     # Display change
26     plt.figure(figsize=(8,6))
27     plt.imshow(ndvi_change, cmap='bwr', vmin=-1, vmax=1)
28     plt.colorbar(label='NDVI Change (2024 - 2015)')
29     plt.title("NDVI Change Detection")
30     plt.axis('off')
31     plt.show()

```

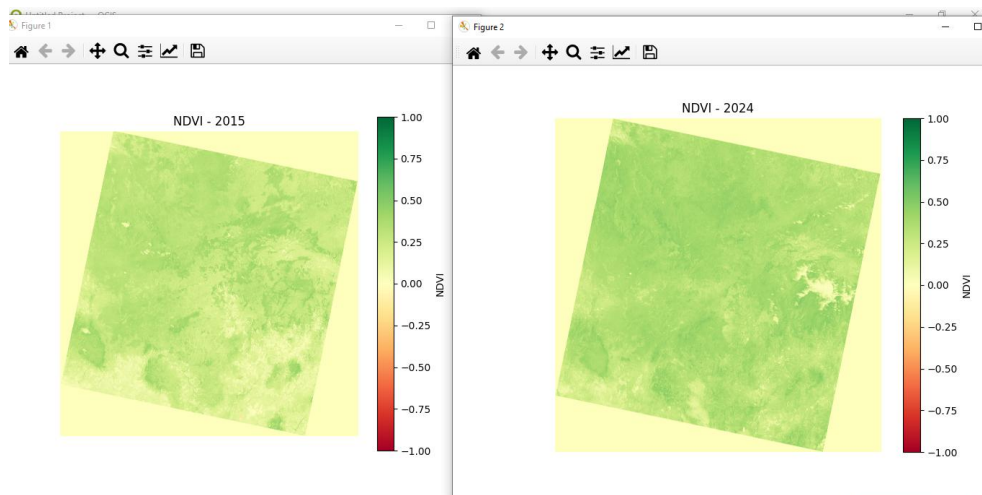


Figure: NDVI (2015&2024)

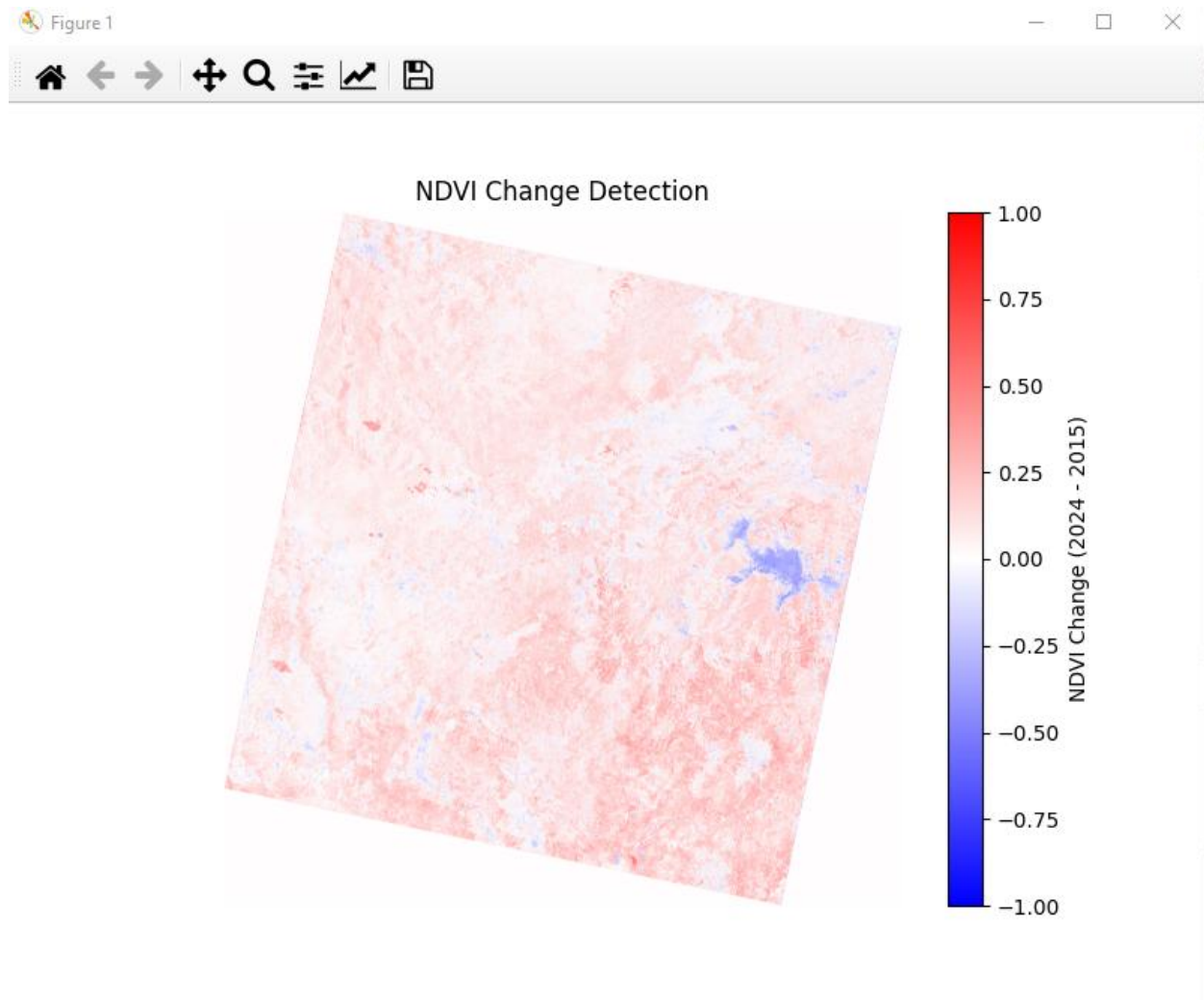


Figure: Change detection

Site suitability analysis